Índice del Proyecto

Documentación del Dataset – Clasificación de Flexiones por Técnica

1. Introducción al problema

La correcta ejecución de los ejercicios de fuerza —especialmente los ejercicios de autocarga como las flexiones— es fundamental para evitar lesiones y garantizar un entrenamiento eficaz. Sin embargo, muchas personas sin experiencia previa desconocen si están realizando el movimiento con una técnica adecuada.

Este proyecto aborda el siguiente problema:

¿Es posible clasificar vídeos de flexiones en función de si la técnica de ejecución es buena o mala?

El objetivo final es desarrollar un sistema inteligente capaz de analizar vídeos de personas realizando flexiones y determinar si la postura corporal es correcta. Esto tendría aplicaciones directas en sistemas de asistencia deportiva, plataformas de entrenamiento remoto y herramientas educativas para principiantes.

---

2. Obtención y procesamiento del conjunto de datos

2.1. Origen de los datos

Hemos obtenido el dataset con los vídeos de la página:
https://www.kaggle.com/datasets/mohamadashrafsalama/pushup/data.
En ellos, varias personas realizan flexiones de dos tipos:

- **Flexiones con técnica correcta**

- **Flexiones con técnica incorrecta**
  (por ejemplo: codos demasiado abiertos, cadera hundida, rango de movimiento insuficiente, etc.)

Cada vídeo se etiqueta manualmente según la calidad de ejecución de la flexión.

2.2. Preprocesamiento con MediaPipe Pose

Para evitar trabajar directamente con frames completos (lo que implicaría un coste computacional elevado), se aplica un preprocesamiento basado en **MediaPipe Pose**, una librería de visión por computador que identifica automáticamente 33 puntos clave del cuerpo (*landmarks*) en cada frame del vídeo.

Para cada frame se extraen:

- Coordenada **x**
- Coordenada **y**

de cada punto clave relevante.

Esto produce un array NumPy por vídeo con la forma (num_frames, 66), donde 66 corresponde a **33 puntos × 2 coordenadas**.

2.3. Cálculo de ángulos articulares

A partir de los landmarks se calcula, para cada frame, un conjunto de **ángulos articulares determinantes**, como por ejemplo:

- Ángulo del codo (izquierdo y derecho)
- Ángulo de la rodilla (izquierdo y derecho)
- Ángulo de la cadera respecto al tronco
- Ángulos relacionados con la posición del torso

Esto genera el dataset final, con cada frame representado por un array de $n_{\text{frames}} \times 8$, donde cada fila contiene los **8 ángulos** calculados para un frame.

Este formato es mucho más manejable y captura la postura corporal de forma eficaz para el posterior modelado.

---

3. Tarea y uso previsto de los datos

El dataset se utilizará para entrenar un modelo de *deep learning* secuencial basado en **redes LSTM**, capaz de analizar la evolución temporal de los ángulos articulares durante la ejecución del ejercicio.

El objetivo de la red es clasificar cada secuencia (cada vídeo) como:

- **Flexión bien ejecutada**

- **Flexión mal ejecutada**

La tarea es, por tanto, un problema de **clasificación binaria sobre datos temporales**.

Librerías

In [1]:
```
import sys
sys.path.append("../src")
from utils import extract_xy_sequence, extract_frame_from_video, draw_landmarks_

import cv2
import os

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
```

Extracción de los puntos clave de los vídeos

Utilizamos la librería Media Pipe para la extracción de las coordenadas de los puntos clave del cuerpo en el vídeo. En el fichero utils.py, tenemos algunas funciones necesarias para la extracción de los landmarks.

El propósito de este proyecto no se centra en la predicción de las coordenadas de los landmarks, sino en analizar como la posición de éstos evoluciona en el vídeo para determinar si la flexión está bien hecha o no.

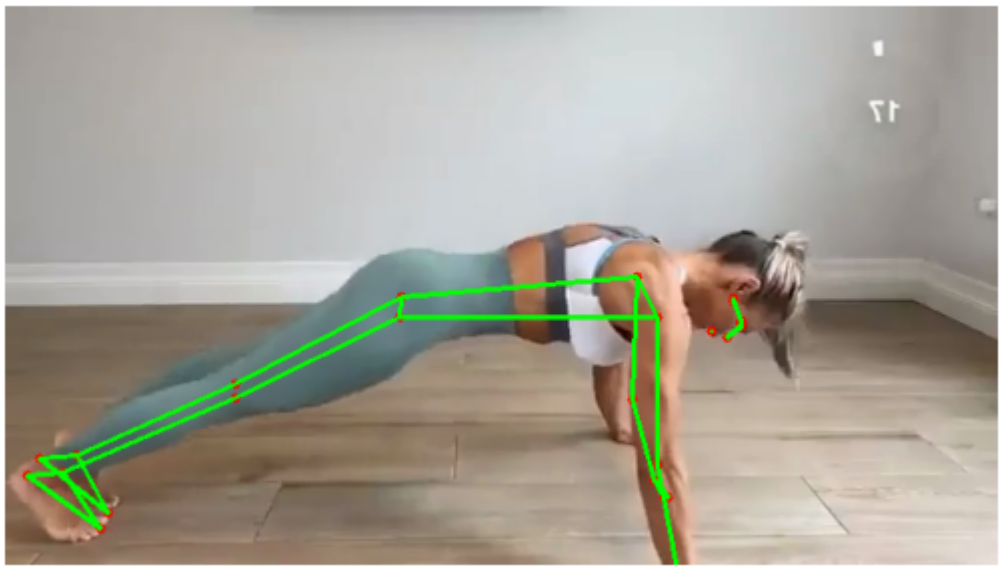
In [2]:
```
# Ejemplo de uso
data_path = "../data"
correct_push_ups_path = os.path.join(data_path, "Correct sequence")
wrong_push_ups_path = os.path.join(data_path, "Wrong sequence")
example_video_path = os.path.join(correct_push_ups_path, "Copy of push up 1.mp4"
seq = extract_xy_sequence(example_video_path)
print("Forma de la secuencia de landmarks", seq.shape)    # → (num_frames, 33, 2
print("Tipo de datos de la secuencia de landmarks", type(seq))          # array
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to S
TDERR
I0000 00:00:1765559400.102212   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
W0000 00:00:1765559400.205271   40901 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559400.214697   40901 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
/opt/miniconda3/envs/exml-py310/lib/python3.10/site-packages/google/protobuf/symb
ol_database.py:55: UserWarning: SymbolDatabase.GetPrototype() is deprecated. Plea
se use message_factory.GetMessageClass() instead. SymbolDatabase.GetPrototype() w
ill be removed soon.
  warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please '
```

```
Forma de la secuencia de landmarks (61, 66)
Tipo de datos de la secuencia de landmarks <class 'numpy.ndarray'>
```

In [3]:
```python
import matplotlib.pyplot as plt
import os

# Ejemplo de uso (comentado):
# frame = cv2.imread('ruta_a_imagen.jpg')  # BGR

# Extraemos un frame del vídeo
path_video = os.path.join(data_path, "Correct sequence", "Copy of push up 1.mp4")
frame10 = extract_frame_from_video(path_video, frame_idx=10)

# Nos quedamos con los landmarks del frame 10
lm = seq[10]  # si seq tiene shape (num_frames, 66) o (num_frames, 33, 2)

# Dibujamos los landmarks sobre el frame
draw_landmarks_on_frame(frame10, lm)
plt.imshow(cv2.cvtColor(frame10, cv2.COLOR_BGR2RGB))
plt.axis('off')

frame10 = extract_frame_from_video(path_video, frame_idx=10)

# Nos quedamos con los landmarks del frame 10
lm = seq[10]  # si seq tiene shape (num_frames, 66) o (num_frames, 33, 2)

# Dibujamos los landmarks sobre el frame
draw_landmarks_on_frame(frame10, lm)
plt.imshow(cv2.cvtColor(frame10, cv2.COLOR_BGR2RGB))
plt.axis('off')
```

Out[3]:
```
(-0.5, 639.5, 359.5, -0.5)
```



In [4]:
```python
# Probamos a añadir padding a la secuencia de ejemplo
PADDING_LENGTH = 150  # longitud máxima de secuencia (frames)

padding_needed = PADDING_LENGTH - seq.shape[0]
padding_array = np.zeros((padding_needed, 66))

seq_padded = np.vstack((seq, padding_array))
print(seq_padded.shape)  # → (150, 66)
```

```
(150, 66)
```

In [5]:
```python
# Extraemos todos los landmarks de los vídeos de flexiones
correct_push_ups_videos = os.listdir(correct_push_ups_path)
wrong_push_ups_videos = os.listdir(wrong_push_ups_path)

# Listas donde almacenaremos las secuencias de landmarks de cada vídeo
correct_push_ups_data = []
wrong_push_ups_data = []

for video_file in correct_push_ups_videos:
    video_path =  os.path.join(correct_push_ups_path, video_file)
    seq = extract_xy_sequence(video_path)

    correct_push_ups_data.append(seq)

for video_file in wrong_push_ups_videos:
    video_path =  os.path.join(wrong_push_ups_path, video_file)
    seq = extract_xy_sequence(video_path)

    wrong_push_ups_data.append(seq)

lengths_correct = np.array(list(map(lambda x: x.shape, correct_push_ups_data)))
lengths_wrong = np.array(list(map(lambda x: x.shape, wrong_push_ups_data)))
```

```
I0000 00:00:1765559401.432676   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559401.525455   40962 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559401.533245   40962 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559402.874675   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559402.963240   40991 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559402.968982   40995 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559404.370606   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559404.458321   41015 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559404.464850   41015 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559405.977036   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559406.065882   41047 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559406.073298   41047 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559407.420880   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559407.512463   41071 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559407.518744   41071 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559409.736014   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559409.823480   41102 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559409.829304   41102 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559411.417381   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559411.506957   41135 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559411.513290   41143 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559413.018902   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559413.105569   41158 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
W0000 00:00:1765559413.111759   41158 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559415.521671   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559415.612061   41188 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559415.618105   41188 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559416.995301   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559417.082905   41217 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559417.089241   41225 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559418.121546   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559418.208960   41251 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559418.214826   41251 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559420.939639   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559421.030288   41298 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559421.036797   41297 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559422.750841   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559422.840241   41341 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559422.846572   41341 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559423.779407   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559423.883031   41380 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559423.889519   41380 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559424.786661   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559424.874256   41403 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559424.880725   41403 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
```

```
I0000 00:00:1765559426.863328   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559426.953320   41454 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559426.959686   41454 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559429.311179   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559429.402755   41487 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559429.408794   41492 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559431.780167   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559431.869160   41520 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559431.875238   41520 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559432.860388   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559432.947920   41544 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559432.954182   41544 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559433.996521   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559434.087474   41569 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559434.093192   41573 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559436.075176   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559436.162285   41591 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559436.168795   41591 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559437.851060   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559437.940121   41617 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559437.946541   41617 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559440.542979   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559440.631798   41663 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
W0000 00:00:1765559440.637460   41663 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559441.891475   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559441.978862   41688 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559441.984765   41688 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559442.963654   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559443.052944   41715 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559443.059149   41708 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559443.999132   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559444.112409   41729 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559444.119222   41729 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559445.993691   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559446.080924   41764 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559446.087254   41766 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559448.298609   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559448.387567   41814 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559448.394034   41810 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559450.354324   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559450.444801   41840 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559450.451664   41842 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559453.108062   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559453.195786   41870 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559453.201874   41878 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
```

```
I0000 00:00:1765559454.817704   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559454.906717   41895 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559454.913493   41895 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559456.188787   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559456.276291   41922 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559456.282275   41922 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559457.510844   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559457.601890   41946 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559457.607664   41952 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559459.242416   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559459.328994   41977 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559459.335446   41980 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559462.100875   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559462.189710   42014 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559462.196072   42014 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559463.782135   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559463.869802   42040 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559463.875557   42040 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559465.535759   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559465.622166   42061 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559465.628415   42065 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559466.629133   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559466.718143   42091 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
W0000 00:00:1765559466.724506   42091 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559468.124665   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559468.211782   42129 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559468.217945   42129 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559468.986778   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559469.074517   42155 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559469.081370   42155 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559470.762277   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559470.850838   42190 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559470.857714   42190 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559472.293150   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559472.383263   42211 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559472.389867   42211 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559474.003793   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559474.093656   42248 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559474.100018   42247 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559475.515746   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559475.603748   42272 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559475.610079   42272 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559476.644139   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559476.732000   42296 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559476.739200   42296 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
```

```
I0000 00:00:1765559478.076074   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559478.175164   42330 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559478.180886   42329 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559479.445698   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559479.533695   42357 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559479.539399   42365 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559482.159427   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559482.248017   42407 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559482.254311   42407 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559483.542284   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559483.643005   42429 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559483.649186   42429 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559485.011199   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559485.101452   42456 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559485.108039   42458 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559486.239029   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559486.348079   42478 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559486.354638   42478 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559490.535999   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559490.625108   42530 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559490.631726   42530 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559492.505675   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559492.594414   42554 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
W0000 00:00:1765559492.600415   42554 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559494.612810   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559494.701515   42586 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559494.707760   42588 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559498.386944   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559498.472150   42607 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559498.478035   42607 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559502.523736   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559502.612141   42651 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559502.618644   42655 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559504.132586   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559504.218880   42674 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559504.224972   42674 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559505.439730   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559505.537551   42708 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559505.543809   42708 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559507.552271   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559507.638784   42723 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559507.644838   42723 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559509.084375   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559509.174578   42768 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559509.181073   42768 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
```

```
I0000 00:00:1765559514.015879   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559514.103421   42805 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559514.109641   42805 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559516.756891   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559516.847283   42842 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559516.853734   42842 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559518.617967   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559518.707907   42870 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559518.715102   42870 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559521.730132   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559521.817832   42914 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559521.824715   42914 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559526.154253   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559526.244941   42966 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559526.251585   42966 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559528.466283   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559528.554799   42987 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559528.561321   42987 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559532.323538   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559532.409857   43040 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559532.416525   43040 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559533.856433   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559533.945113   43067 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
W0000 00:00:1765559533.951461   43067 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559541.063038   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559541.152394   43127 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559541.158562   43127 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559543.000839   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559543.088437   43171 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559543.094557   43171 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559544.710658   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559544.801395   43199 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559544.807815   43199 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559547.433409   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559547.521499   43227 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559547.528054   43235 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559549.704276   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559549.793534   43265 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559549.800075   43264 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559555.612466   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559555.717052   43336 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559555.722911   43339 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
```

```
Warning: No se detectaron landmarks en un frame de ../data/Wrong sequence/Copy of
push up 153.mp4
```

```
I0000 00:00:1765559556.858211   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559556.947028   43367 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559556.953745   43367 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559558.800024   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559558.888484   43393 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559558.894713   43393 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559560.601326   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559560.691050   43427 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559560.697260   43434 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559561.850138   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559561.938481   43452 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559561.944350   43452 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559563.040501   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559563.144486   43471 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559563.150499   43471 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559565.290883   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559565.377692   43531 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559565.383929   43531 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559567.117888   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559567.211147   43557 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559567.216886   43557 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559568.298803   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559568.402024   43579 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

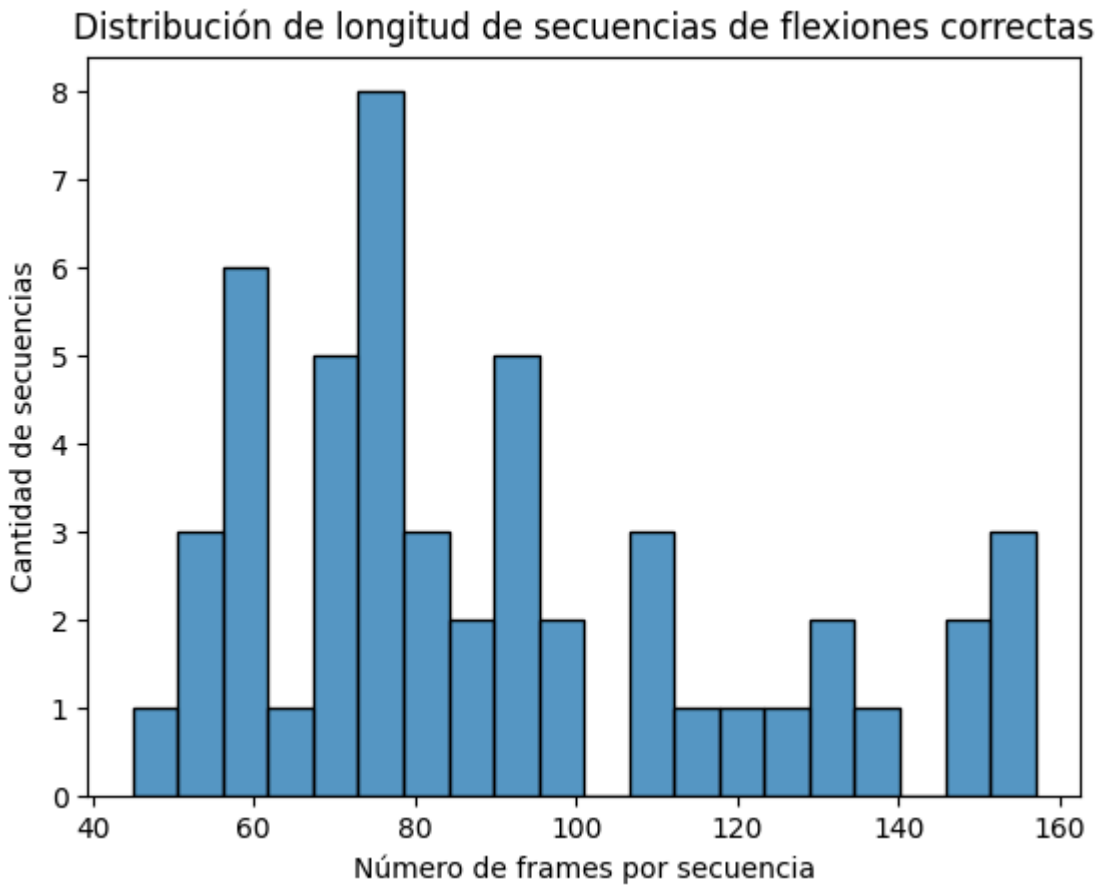
```
feedback tensors.
W0000 00:00:1765559568.408346   43578 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559569.434338   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559569.522002   43602 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559569.528512   43602 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559570.990366   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559571.075902   43630 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559571.082112   43639 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559573.233725   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559573.324753   43654 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559573.331081   43661 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559574.874230   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559574.960987   43681 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559574.967852   43681 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559576.385963   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559576.472767   43715 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559576.478855   43715 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559577.283163   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559577.373041   43748 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559577.379487   43747 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559579.450908   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559579.539122   43782 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559579.545123   43782 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
```

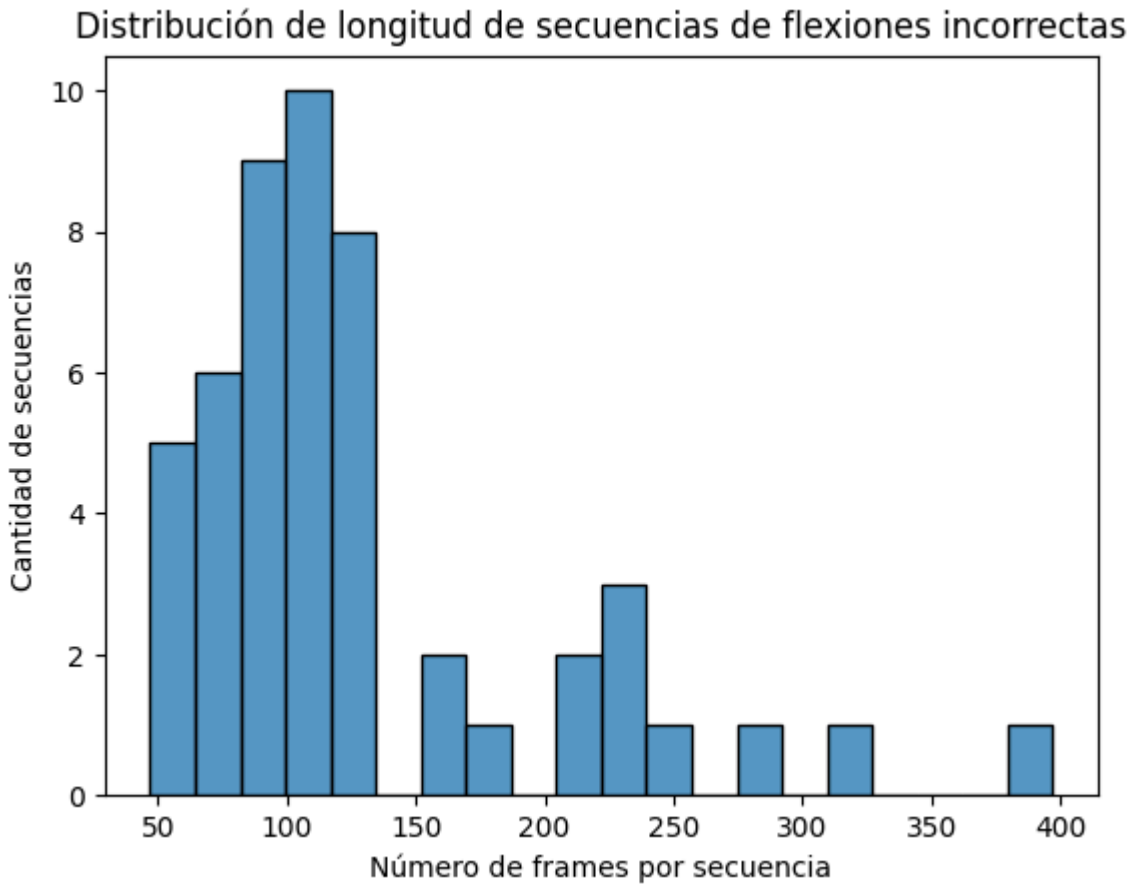
```
I0000 00:00:1765559581.622049   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559581.708813   43812 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559581.715236   43816 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559582.842034   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559582.945416   43853 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559582.951905   43857 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559584.644193   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559584.732651   43879 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559584.738811   43879 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559586.468592   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559586.555114   43906 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559586.561807   43906 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559588.501158   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559588.588489   43933 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559588.594361   43935 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559590.588115   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559590.676173   43974 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559590.682360   43974 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559592.275495   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559592.360844   43996 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559592.367260   43996 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559593.854666   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559593.942220   44021 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
W0000 00:00:1765559593.948093   44020 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559595.066457   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559595.155417   44043 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559595.161166   44044 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559596.530009   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559596.618651   44068 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559596.625161   44068 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559598.667751   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559598.757262   44124 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559598.763947   44124 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
```

In [6]:

```python
# Graficamos la distribución de número de frames por secuencia
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot([l[0] for l in lengths_correct], bins=20)
plt.xlabel("Número de frames por secuencia")
plt.ylabel("Cantidad de secuencias")
plt.title("Distribución de longitud de secuencias de flexiones correctas")
plt.show()

sns.histplot([l[0] for l in lengths_wrong], bins=20)
plt.xlabel("Número de frames por secuencia")
plt.ylabel("Cantidad de secuencias")
plt.title("Distribución de longitud de secuencias de flexiones incorrectas")
plt.show()
```





Podemos observar que la gran mayoría de vídeos no tiene más de 200 frames. Por ende, sería conveniente añadirle un padding de 0s a los que tenga menos frames y truncar aquellos que superen esa cantidad.

In [7]:
```python
import os
import numpy as np

def load_landmark_sequences_from_folder(folder_path, padding_length=200):
    """
    Procesa todos los videos de una carpeta y devuelve un array
    de secuencias de landmarks con shape (n_videos, padding_length, 66).

    Args:
        folder_path (str): ruta a la carpeta con vídeos
        padding_length (int): nº máximo de frames para pad/recorte

    Returns:
        np.ndarray: array final con shape (n_videos, padding_length, 66)
    """

    videos = os.listdir(folder_path)
    sequences = []

    for video_name in videos:
        video_path = os.path.join(folder_path, video_name)

        # Extraemos coordenadas (num_frames, 33, 2)
        seq = extract_xy_sequence(video_path)

        # Aplanamos por frame → (num_frames, 66)
        seq = seq.reshape(seq.shape[0], -1)

        # === Padding o recorte ===
        n_frames = seq.shape[0]

        if n_frames < padding_length:
            # Padding con ceros
            padding_needed = padding_length - n_frames
            padding = np.zeros((padding_needed, 66))
            seq_padded = np.vstack((seq, padding))

        elif n_frames > padding_length:
            # Recorte
            seq_padded = seq[:padding_length, :]

        else:
            seq_padded = seq

        # Añadimos al dataset
        sequences.append(seq_padded)

    # Convertimos lista → numpy array 3D
    return np.array(sequences)
```

In [8]:
```python
import math

frames_videos_correctos = lengths_correct[:, 0]
frames_videos_incorrectos = lengths_wrong[:, 0]

# Concatenamos
longitudes_de_secuencia = np.concatenate([frames_videos_correctos, frames_videos

# Calculamos el valor L_opt con percentil 90 y redondeamos al entero superior má
```

```
percentil_de_corte = 90
L_opt_flotante = np.percentile(longitudes_de_secuencia, percentil_de_corte)
L_opt = math.ceil(L_opt_flotante)

print(f"El valor L_opt sugerido es: {L_opt} frames")
```

```
El valor L_opt sugerido es: 160 frames
```

In [9]:
```
# Procesamos todos los landmarks de los vídeos de flexiones en ambas carpetas
PADDING_LENGTH = 160  # Longitud máxima de secuencia (frames)

correct_push_ups_data = load_landmark_sequences_from_folder(correct_push_ups_pat
wrong_push_ups_data   = load_landmark_sequences_from_folder(wrong_push_ups_path,


print("Forma de los landmarks arrays de los vídeos de flexiones correctas:", cor
print("Forma de los landmarks arrays de los vídeos de flexiones incorrectas:", w
```

```
I0000 00:00:1765559603.013633   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559603.104807   44177 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559603.111145   44177 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
/opt/miniconda3/envs/exml-py310/lib/python3.10/site-packages/google/protobuf/symb
ol_database.py:55: UserWarning: SymbolDatabase.GetPrototype() is deprecated. Plea
se use message_factory.GetMessageClass() instead. SymbolDatabase.GetPrototype() w
ill be removed soon.
  warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please '
I0000 00:00:1765559604.497311   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559604.584653   44198 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559604.590981   44198 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559605.973246   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559606.060884   44229 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559606.067527   44238 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559607.547060   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559607.633871   44252 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559607.640116   44255 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559609.000307   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559609.100917   44280 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559609.109173   44280 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559611.363371   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559611.450499   44317 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559611.457153   44317 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559613.045553   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559613.132611   44349 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559613.138880   44349 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
I0000 00:00:1765559614.647321   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559614.738523   44374 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559614.744947   44377 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559617.189287   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559617.277262   44402 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559617.283287   44402 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559618.639456   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559618.726656   44428 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559618.733233   44428 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559619.785191   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559619.884111   44457 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559619.893081   44457 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559622.655910   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559622.744770   44494 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559622.751062   44494 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559624.446706   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559624.532194   44534 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559624.538686   44533 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559625.459552   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559625.547541   44561 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559625.553802   44569 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559626.476582   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559626.565114   44586 inference_feedback_manager.cc:114] Feedback
```

```
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559626.571641   44586 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559628.565578   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559628.656885   44617 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559628.663759   44617 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559631.143300   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559631.231469   44652 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559631.237649   44652 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559633.637195   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559633.726431   44709 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559633.733126   44711 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559634.770141   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559634.859397   44732 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559634.865642   44732 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559635.933480   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559636.021191   44767 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559636.027953   44767 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559638.058662   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559638.148098   44788 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559638.154794   44788 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559639.842708   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559639.930883   44817 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559639.937601   44817 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
I0000 00:00:1765559642.549590   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559642.638444   44852 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559642.645170   44852 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559643.926547   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559644.029882   44880 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559644.037791   44880 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559645.027441   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559645.115809   44901 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559645.122165   44901 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559646.056037   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559646.145271   44923 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559646.151745   44923 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559647.976252   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559648.066384   44952 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559648.073357   44952 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559650.264818   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559650.353877   44988 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559650.360449   44988 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559652.276912   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559652.367089   45024 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559652.373291   45024 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559654.982727   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559655.070448   45067 inference_feedback_manager.cc:114] Feedback
```

```
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559655.076232   45067 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559656.665708   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559656.754267   45101 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559656.760478   45101 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559657.990655   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559658.076951   45124 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559658.083262   45123 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559659.282345   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559659.370063   45158 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559659.376633   45158 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559660.994629   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559661.083047   45189 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559661.089353   45181 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559663.823545   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559663.909907   45212 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559663.916152   45220 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559665.451651   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559665.538903   45236 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559665.544860   45243 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559667.174791   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559667.260574   45262 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559667.266993   45262 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
I0000 00:00:1765559668.257040   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559668.344082   45300 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559668.350140   45300 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559669.726553   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559669.814240   45332 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559669.819877   45331 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559670.589587   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559670.673881   45356 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559670.680394   45356 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559672.272876   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559672.360741   45381 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559672.366987   45381 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559673.704498   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559673.793308   45408 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559673.799369   45408 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559675.391696   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559675.481455   45437 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559675.487808   45437 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559676.890480   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559676.977860   45463 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559676.984123   45463 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559678.013625   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559678.100617   45503 inference_feedback_manager.cc:114] Feedback
```

```
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559678.106739   45503 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559679.442492   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559679.528343   45537 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559679.534899   45538 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559680.788919   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559680.874964   45568 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559680.881521   45568 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559683.520043   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559683.604920   45594 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559683.611226   45594 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559684.895379   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559684.981308   45643 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559684.987438   45643 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559686.313027   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559686.396894   45666 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559686.403056   45674 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559687.465078   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559687.570859   45687 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559687.576718   45687 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559691.568267   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559691.655654   45740 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559691.662044   45740 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
I0000 00:00:1765559693.498885   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559693.587087   45767 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559693.592908   45767 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559695.634061   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559695.719759   45802 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559695.726667   45802 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559699.529422   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559699.616490   45847 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559699.622428   45847 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559703.779760   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559703.868381   45887 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559703.874677   45887 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559705.413413   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559705.500932   45912 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559705.507980   45912 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559706.755103   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559706.845317   45959 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559706.851672   45959 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559708.906076   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559708.992244   45992 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559708.998552   45992 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559710.489372   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559710.577694   46033 inference_feedback_manager.cc:114] Feedback
```

```
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559710.584453   46039 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559715.595235   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559715.684100   46067 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559715.690674   46067 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559718.430103   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559718.519116   46133 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559718.526249   46137 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559720.335075   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559720.421094   46168 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559720.427301   46168 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559723.403964   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559723.491522   46207 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559723.498136   46207 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559727.764552   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559727.855293   46267 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559727.861214   46267 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559730.016375   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559730.105993   46301 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559730.112001   46301 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559733.839706   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559733.924578   46350 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559733.930667   46350 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
I0000 00:00:1765559735.381966   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559735.470220   46371 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559735.476300   46379 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559742.425360   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559742.514033   46456 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559742.519972   46456 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559744.372239   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559744.460966   46496 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559744.466614   46497 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559746.054710   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559746.142473   46533 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559746.148364   46541 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559748.834481   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559748.923481   46567 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559748.930037   46567 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559751.090747   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559751.180799   46607 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559751.186555   46607 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559757.148135   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559757.239743   46656 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559757.246059   46660 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
Warning: No se detectaron landmarks en un frame de ../data/Wrong sequence/Copy of
push up 153.mp4
```

```
I0000 00:00:1765559758.408221   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559758.498066   46714 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559758.505011   46714 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559760.383489   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559760.473925   46757 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559760.481572   46757 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559762.247468   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559762.334694   46780 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559762.341467   46780 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559763.522315   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559763.617102   46803 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559763.623416   46803 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559764.713904   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559764.817320   46832 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559764.823552   46835 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559766.960327   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559767.045919   46867 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559767.051896   46867 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559768.748424   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559768.841481   46902 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559768.847810   46902 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559769.921376   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559770.024826   46929 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
W0000 00:00:1765559770.031627   46929 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559771.065723   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559771.152727   46958 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559771.158920   46963 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559772.625410   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559772.713330   46983 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559772.720120   46983 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559774.902458   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559774.993303   47016 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559774.999720   47016 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559776.555298   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559776.642296   47070 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559776.648913   47078 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559778.064089   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559778.157181   47113 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559778.163429   47113 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559778.965757   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559779.051366   47147 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559779.057382   47147 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559781.095752   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559781.181938   47181 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559781.188293   47188 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
```

```
I0000 00:00:1765559783.219553   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559783.305986   47210 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559783.312036   47210 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559784.438188   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559784.552328   47234 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559784.558894   47241 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559786.335626   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559786.424331   47272 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559786.431187   47278 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559788.210970   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559788.313434   47297 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559788.321195   47297 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559790.319248   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559790.405353   47346 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559790.411988   47346 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559792.469424   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559792.562608   47380 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559792.569317   47382 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559794.224130   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559794.312620   47407 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559794.319310   47407 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559795.889666   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559795.984645   47435 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
```

```
feedback tensors.
W0000 00:00:1765559795.991463   47435 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559797.159394   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559797.251623   47457 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559797.258403   47457 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559798.647421   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559798.737366   47500 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559798.744307   47500 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
I0000 00:00:1765559800.792335   29611 gl_context.cc:357] GL version: 2.1 (2.1 Met
al - 90.5), renderer: Apple M1 Pro
W0000 00:00:1765559800.884133   47550 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
W0000 00:00:1765559800.891006   47553 inference_feedback_manager.cc:114] Feedback
manager requires a model with a single signature inference. Disabling support for
feedback tensors.
Forma de los landmarks arrays de los vídeos de flexiones correctas: (50, 160, 66)
Forma de los landmarks arrays de los vídeos de flexiones incorrectas: (50, 160, 6
6)
```

In [10]:
```
# Almacenamos en discos los arrays procesados
np.save("../data/processed/correct_push_ups_landmarks.npy", correct_push_ups_dat
np.save("../data/processed/wrong_push_ups_landmarks.npy",   wrong_push_ups_data)
```

Análisis exploratorio de datos

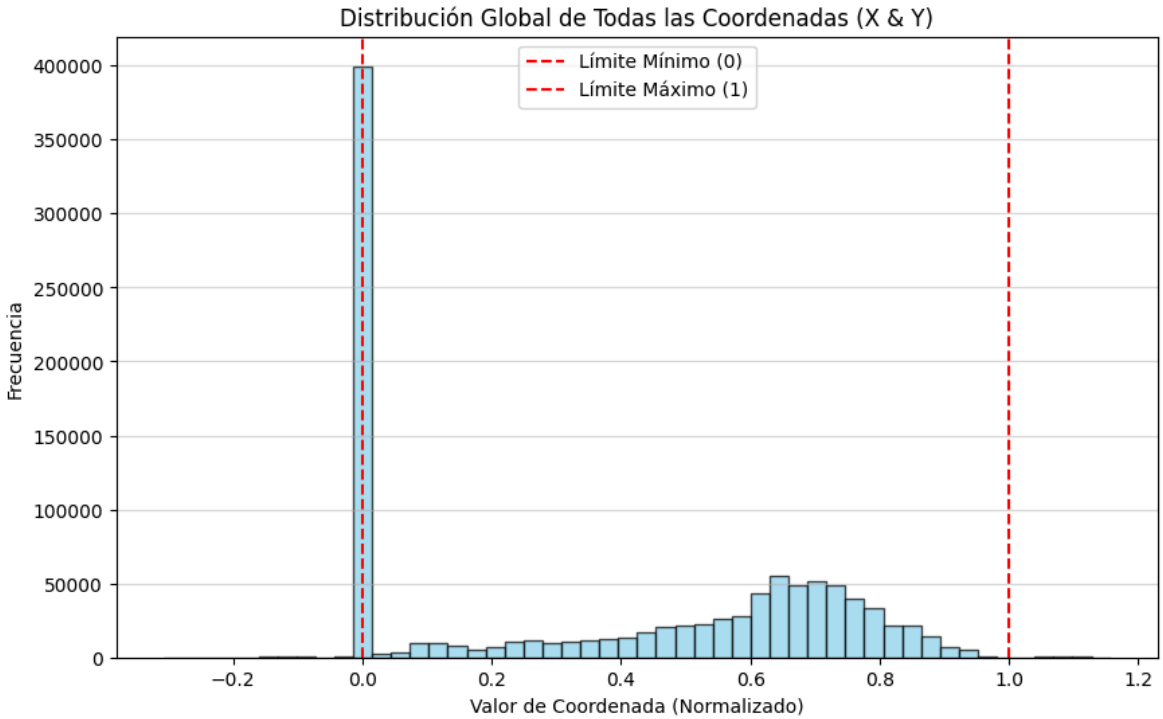
In [11]:
```
import numpy as np
import os

# Indicamos la ruta a los datos
PATH = "../Data/processed"

# Cargamos los datos
# Contienen etiquetas con los puntos clave del cuerpo durante las flexiones
# Los arrays tienen forma (num_ejemplos, num_frames, num_keypoints)
# num_keypoints -> 1er_punto_clave_x, 1er_punto_clave_y, 2º_punto_clave_x, 2º_pu

correct_push_ups = np.load(os.path.join(PATH, 'correct_push_ups_landmarks.npy'))
wrong_push_ups = np.load(os.path.join(PATH, 'wrong_push_ups_landmarks.npy'))

print("Correct push-ups shape:", correct_push_ups.shape)
print("Incorrect push-ups shape:", wrong_push_ups.shape)
```

```
Correct push-ups shape: (50, 160, 66)
Incorrect push-ups shape: (50, 160, 66)
```

Vemos que ambos conjuntos de datos tienen el mismo número de videos, cada vídeo tiene el mismo número de frames y cada frame tiene el mismo número de landmarks.

¿Cuál es el rango de valores que toman las coordenadas?

In [12]:
```
import numpy as np
import matplotlib.pyplot as plt

# Asumiendo que 'data_array' es un numpy array de forma (50, 160, 66)
# data_array = np.load('tu_archivo.npy')

# 1. Aplana todos los datos de coordenadas
all_coords = np.concatenate((correct_push_ups, wrong_push_ups)).flatten()

# 2. Configura y graficar el histograma
plt.figure(figsize=(10, 6))
plt.hist(all_coords, bins=50, color='skyblue', edgecolor='black', alpha=0.7)

plt.title('Distribución Global de Todas las Coordenadas (X & Y)')
plt.xlabel('Valor de Coordenada (Normalizado)')
plt.ylabel('Frecuencia')
plt.grid(axis='y', alpha=0.5)
plt.axvline(x=0, color='r', linestyle='--', label='Límite Mínimo (0)')
plt.axvline(x=1, color='r', linestyle='--', label='Límite Máximo (1)')
plt.legend()
plt.show()
```



La herramienta de visión artificial MediaPipe Pose predice la posición de 33 puntos clave del cuerpo (x, y), tomando como origen la esquina inferior izquierda del frame. Cada vídeo tenía una longitud distinta originalmente. Sin embargo, anteriormente añadimos paddings de 0s para que todas las secuencias tuvieran la misma longitud, lo que explica la gran cantidad de ceros observada en el histograma. En la arquitectura de red indicaremos que estos 0s son de padding con una capa Masking.

Por otro lado, algunos valores de las coordenadas superan 1 o son menores a 0. Esto ocurre cuando ciertos puntos clave quedan fuera del plano de la imagen. Estos datos pueden considerarse válidos, ya que únicamente representan posiciones fuera del marco visual y, siempre que mantengan coherencia geográfica con el resto del cuerpo, no deberían afectar negativamente al modelo de clasificación.

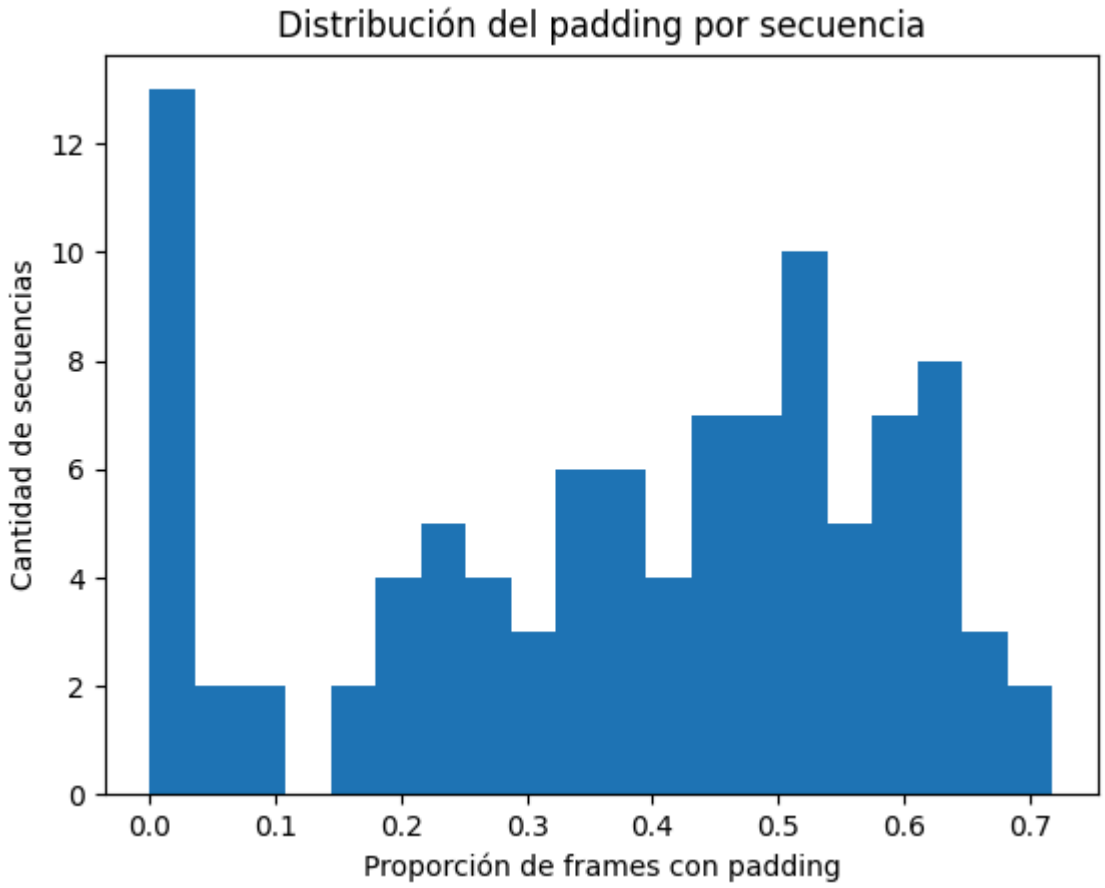
¿Cuál es la proporción de relleno (padding) en cada vídeo?

Veámos la distribución de esta medida en el dataset.

In [13]:
```
video0 = correct_push_ups[0]  # primer vídeo
#print(video0.shape)  # (160, 66)
#print(video0)        # imprime todos los frames y landmarks del vídeo 0, cada f

frame0_video0 = correct_push_ups[0, 0]  # primer frame del primer vídeo
#print(frame0_video0.shape)  # (66,)
#print(frame0_video0)        # imprime los 66 landmarks de ese frame (coincide c

landmark0_frame0_video0 = correct_push_ups[0, 0, 0]  # primer landmark
#print(landmark0_frame0_video0)  # valor del landmark
```

In [14]:
```
# Calculamos la proporción de frames vacíos (rellenos con ceros) por secuencia
padding_ratios = []

data = np.vstack((correct_push_ups, wrong_push_ups))
for seq in data:
    # Un frame está vacío si todos sus landmarks son 0
    empty_frames = np.sum(np.all(seq == 0, axis=1)) # suma cada fila que es todo
    ratio = empty_frames / seq.shape[0]
    padding_ratios.append(ratio)

# Estadísticas del padding
print(f"Media de padding por secuencia: {np.mean(padding_ratios):.2%}")
print(f"Máximo padding: {np.max(padding_ratios):.2%}")
print(f"Mínimo padding: {np.min(padding_ratios):.2%}")

# Distribución visual
plt.hist(padding_ratios, bins=20)
plt.xlabel("Proporción de frames con padding")
plt.ylabel("Cantidad de secuencias")
plt.title("Distribución del padding por secuencia")
plt.show()
```

```
Media de padding por secuencia: 37.63%
Máximo padding: 71.88%
Mínimo padding: 0.00%
```



El porcentaje obtenido es un poco alto, 37.63% pero esto se debe a que en los videos incorrectos, algunos (9 videos) superaban los 200 frames, llegando hasta los 400, por lo que esto incrementó el numero óptimo de frames por vídeo al calcular L_{opt}.

Vamos a comprobar si los landmarks están normalizados

In [15]:
```
longitudes_combinadas = np.vstack((correct_push_ups, wrong_push_ups)).reshape(-1
# Con el -1 del reshape combinamos el número de videos y el número de frames, ob
# 'correct_push_ups.shape[2]' lo que hace es mantener el número de landmarks (66

min_val = np.min(longitudes_combinadas)
max_val = np.max(longitudes_combinadas)

print(f"Valor mínimo de coordenadas: {min_val:.4f}")
print(f"Valor máximo de coordenadas: {max_val:.4f}")
```

```
Valor mínimo de coordenadas: -0.3072
Valor máximo de coordenadas: 1.1568
```

Vamos a normalizar los landmarks.

In [16]:
```
rango = max_val - min_val

# Aplicamos la fórmula de Re-normalización
longitudes_combinadas_escaladas = (longitudes_combinadas - min_val) / rango

correct_push_ups_escaladas = (correct_push_ups - min_val) / rango
incorrect_push_ups_escaladas = (wrong_push_ups - min_val) / rango
```

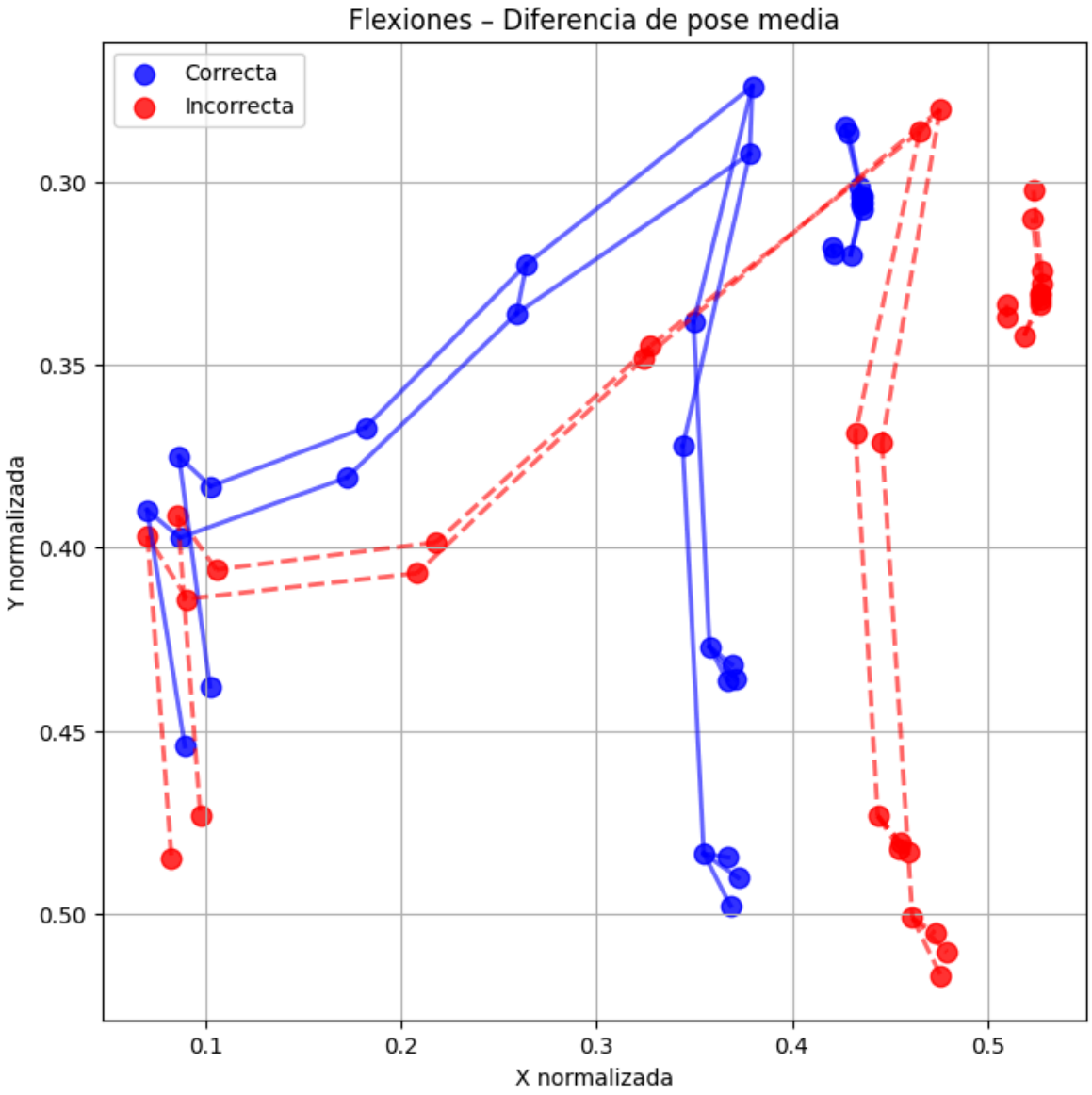
In [17]:
```
correct_push_ups_escaladas = correct_push_ups
incorrect_push_ups_escaladas = wrong_push_ups
```

Podemos ver que en el caso de los vídeos correctos (los únicos que hemos seleccionado), los landmarks se agrupan por zonas más o menos diferenciadas, lo cual es indicativo de que los landmarks están ordenados. Es un poco complicado diferenciar las partes del cuerpo, por lo que vamos a ver la distribución espacial de los landmarks uniendo los puntos para poder visualizar la pose promedio de cada clase.

In [18]:
```
# Definimos las conexiones del esqueleto (pares de índices de landmarks)
# Esto es específico para un modelo de 33 landmarks (como MediaPipe Pose), cada
# Si buscamos información sobre las conexiones de MediaPipe Pose: https://ai.goo
CONNECTIONS = [
    # Cabeza
    (0, 1), (1, 2), (2, 3), (3, 7),
    (0, 4), (4, 5), (5, 6), (6, 8),
    (9, 10),

    # Torso y Hombros
    (11, 12),  # Hombros
    (11, 23),  # Hombro Izquierdo a Cadera Izquierda
    (12, 24),  # Hombro Derecho a Cadera Derecha
    (23, 24),  # Caderas

    # Brazo Izquierdo
    (11, 13), (13, 15),
    # Mano Izquierda
    (15, 17), (15, 19), (15, 21),

    # Brazo Derecho
    (12, 14), (14, 16),
    # Mano Derecha
    (16, 18), (16, 20), (16, 22),

    # Pierna Izquierda
    (23, 25), (25, 27),
    # Pie Izquierdo
    (27, 29), (29, 31),

    # Pierna Derecha
    (24, 26), (26, 28),
    # Pie Derecho
    (28, 30), (30, 32)
]

def plot_pose_media(data_correctas, data_incorrectas, title="Comparación de Pose
    """
    data_correctas: array (N, F, dims)
    data_incorrectas: array (N, F, dims)
    dims = 66 (x,y), 99 (x,y,z), 132 (x,y,z,visibility)

    Dibuja el esqueleto promedio para ambas clases.
    """

    def compute_pose_media(dataset):
        frames = dataset.reshape(-1, dataset.shape[-1])  # Aplanamos (n_videos*f
        dims_per_point = frames.shape[-1] // 33 # Calculamos cuántas dimensiones
```

```
        mean_pose = np.mean(frames, axis=0) # Calculamos el promedio de cada col
        return mean_pose.reshape(33, dims_per_point)[:, :2] # Reorganizamos ese

    pose_correcta = compute_pose_media(data_correctas)
    pose_incorrecta = compute_pose_media(data_incorrectas)

    # ---- GRÁFICO ----
    plt.figure(figsize=(8, 8))
    plt.title(title)
    plt.xlabel("X normalizada")
    plt.ylabel("Y normalizada")
    plt.gca().invert_yaxis()

    # Puntos
    plt.scatter(pose_correcta[:, 0], pose_correcta[:, 1],
                color="blue", label="Correcta", s=80, alpha=0.8)
    plt.scatter(pose_incorrecta[:, 0], pose_incorrecta[:, 1],
                color="red", label="Incorrecta", s=80, alpha=0.8)

    # Conexiones
    for a, b in CONNECTIONS:
        x1, y1 = pose_correcta[a]
        x2, y2 = pose_correcta[b]
        plt.plot([x1, x2], [y1, y2],
                 color="blue", alpha=0.6, linewidth=2)

        x1, y1 = pose_incorrecta[a]
        x2, y2 = pose_incorrecta[b]
        plt.plot([x1, x2], [y1, y2],
                 color="red", alpha=0.6, linewidth=2, linestyle="--")

    plt.legend()
    plt.grid(True)
    plt.show()
```

In [19]:
```
plot_pose_media(correct_push_ups_escaladas,
                incorrect_push_ups_escaladas,
                title="Flexiones – Diferencia de pose media")
```



Se aprecian perfectamente las partes del cuerpo en ambas clases y se ve cómo en la clase de flexiones incorrectas, las rodillas y la cabeza están más abajo de lo normal.

Estadísticas descriptivas

Media, desviación estándar y rango por landmark (x, y).

Comparación entre clases: ¿la media de la posición del pecho o codo difiere entre clases?

In [20]:
```
# Definimos los nombres de los landmarks
LANDMARK_NAMES = [
    'nose', 'left eye (inner)', 'left eye', 'left eye (outer)', 'right eye (inne
    'left ear', 'right ear', 'mouth (left)', 'mouth (right)', 'left shoulder', '
    'right elbow', 'left wrist', 'right wrist', 'left pinky', 'right pinky', 'le
    'left thumb', 'right thumb', 'left hip', 'right hip', 'left knee', 'right kn
    'right ankle', 'left heel', 'right heel', 'left foot index', 'right foot ind
]

def compute_landmark_statistics(dataset, label=None):
    """
    dataset: array (N_videos, N_frames, dims)
    dims = 33*2, 33*3, o 33*4
```

```
    Returns DataFrame con estadísticas descriptivas de cada coordenada.
    """

    n_points = 33
    dims_per_point = dataset.shape[-1] // n_points

    # Generamos nombres de coordenadas dinámicamente
    coord_letters = ["X", "Y", "Z", "visibility"][:dims_per_point]
    coord_names = [f"{lm} ({c})" for lm in LANDMARK_NAMES for c in coord_letters

    # Aplanamos los datos para que cada fila sea un frame único. (N_total_frames
    frames = dataset.reshape(-1, dataset.shape[-1])

    # Calculamos las estadísticas a lo largo del eje 0 (todos los frames)
    stats = {
        "Coordenada": coord_names,
        "Media (μ)": np.mean(frames, axis=0),
        "Desviación Estándar (σ)": np.std(frames, axis=0),
        "Máximo": np.max(frames, axis=0),
        "Mínimo": np.min(frames, axis=0)
    }
    stats["Rango (Max - Min)"] = stats["Máximo"] - stats["Mínimo"]

    df = pd.DataFrame(stats)

    if label is not None:
        df.insert(0, "Clase", label)

    return df
```

In [21]:

```
NUM_COORDS = correct_push_ups_escaladas.shape[2]  # 66 (33 puntos * 2 coordenada
# Análisis de la clase Correcta
df_correctas = compute_landmark_statistics(correct_push_ups_escaladas, label='Co

# Análisis de la clase Incorrecta
df_incorrectas = compute_landmark_statistics(incorrect_push_ups_escaladas, label

# Análisis del conjunto combinado (para variabilidad global)
longitudes_combinadas_escaladas = np.vstack((correct_push_ups_escaladas.reshape(
                                             incorrect_push_ups_escaladas.reshape

df_combinado = compute_landmark_statistics(longitudes_combinadas_escaladas, labe

# Podemos usar pandas.concat para ver todas las estadísticas juntas
df_final = pd.concat([df_correctas, df_incorrectas, df_combinado], ignore_index=
print(df_final.to_markdown(index=False)) # Hacemos que la tabla se vea bien en l
```

Clase	Coordenada	Media (μ)	Desviación Estándar (σ)	Máximo	Mínimo	Rango (Max - Min)
Correcta	nose (X)	0.429982	0.381792	0.940509	0	0.940509
Correcta	nose (Y)	0.319879	0.305345	0.923014	0	0.923014
Correcta	left eye (inner) (X)	0.436074	0.38698	0.942475	0	0.942475
Correcta	left eye (inner) (Y)	0.307315	0.296205	0.920705	0	0.920705
Correcta	left eye (X)	0.43604	0.386923	0.941455	0	0.941455
Correcta	left eye (Y)	0.305673	0.294774	0.918398	0	0.918398
Correcta	left eye (outer) (X)	0.435995	0.386855	0.940449	0	0.940449
Correcta	left eye (outer) (Y)	0.303786	0.293143	0.915451	0	0.915451
Correcta	right eye (inner) (X)	0.435489	0.386517	0.942949	0	0.942949
Correcta	right eye (inner) (Y)	0.306204	0.29558	0.917519	0	0.917519
Correcta	right eye (X)	0.435019	0.386122	0.942341	0	0.942341
Correcta	right eye (Y)	0.303933	0.293846	0.913205	0	0.913205
Correcta	right eye (outer) (X)	0.434483	0.385676	0.941706	0	0.941706
Correcta	right eye (outer) (Y)	0.301355	0.291893	0.908472	0	0.908472
Correcta	left ear (X)	0.429158	0.380826	0.921061	0	0.921061
Correcta	left ear (Y)	0.286464	0.278301	0.875618	0	0.875618
Correcta	right ear (X)	0.427108	0.379317	0.923467	0	0.923467
Correcta	right ear (Y)	0.284662	0.278038	0.868351	0	0.868351
Correcta	mouth (left) (X)	0.421412	0.374307	0.924196	0	0.924196
Correcta	mouth (left) (Y)	0.319527	0.303466	0.903963	0	0.903963
Correcta	mouth (right) (X)	0.420599	0.373678	0.923467	0	0.923467
Correcta	mouth (right) (Y)	0.317641	0.302282	0.898981	0	0.898981
Correcta	left shoulder (X)	0.379677	0.337383	0.847306	0	0.847306
Correcta	left shoulder (Y)	0.273819	0.256757	0.751023	0	0.751023
Correcta	right shoulder (X)	0.378344	0.338654	0.859115	0	0.859115
Correcta	right shoulder (Y)	0.292276	0.275525	0.827523	0	0.827523
Correcta	left elbow (X)	0.34968	0.310773	0.801946	0	0.801946
Correcta	left elbow (Y)	0.338301	0.301151	0.798809	0	0.798809

Correcta	right elbow (X)	0.344139	0.310689	0.803778	0	0.803778
Correcta	right elbow (Y)	0.372165	0.332806	0.92978	0	0.92978
Correcta	left wrist (X)	0.357805	0.317682	0.799056	0	0.799056
Correcta	left wrist (Y)	0.427088	0.379607	0.935731	0	0.935731
Correcta	right wrist (X)	0.35487	0.318368	0.804167	0	0.804167
Correcta	right wrist (Y)	0.48364	0.430915	1.08222	0	1.08222
Correcta	left pinky (X)	0.36927	0.327613	0.810523	0	0.810523
Correcta	left pinky (Y)	0.431987	0.384391	0.972936	0	0.972936
Correcta	right pinky (X)	0.368567	0.330002	0.820667	0	0.820667
Correcta	right pinky (Y)	0.497914	0.444505	1.15679	0	1.15679
Correcta	left index (X)	0.371326	0.329394	0.814191	0	0.814191
Correcta	left index (Y)	0.435714	0.387663	0.982433	0	0.982433
Correcta	right index (X)	0.372502	0.333011	0.82505	0	0.82505
Correcta	right index (Y)	0.490265	0.437552	1.12223	0	1.12223
Correcta	left thumb (X)	0.366695	0.325398	0.810247	0	0.810247
Correcta	left thumb (Y)	0.436251	0.387941	0.982168	0	0.982168
Correcta	right thumb (X)	0.367434	0.328543	0.821377	0	0.821377
Correcta	right thumb (Y)	0.484682	0.432172	1.10793	0	1.10793
Correcta	left hip (X)	0.264208	0.239139	0.704815	0	0.704815
Correcta	left hip (Y)	0.322471	0.29122	0.773461	0	0.773461
Correcta	right hip (X)	0.259141	0.236254	0.702599	0	0.702599
Correcta	right hip (Y)	0.335971	0.303974	0.843163	0	0.843163
Correcta	left knee (X)	0.181947	0.172629	0.610453	0	0.610453
Correcta	left knee (Y)	0.367073	0.326974	0.799709	0	0.799709
Correcta	right knee (X)	0.172447	0.166203	0.592309	0	0.592309
Correcta	right knee (Y)	0.38077	0.339771	0.84561	0	0.84561
Correcta	left ankle (X)	0.102648	0.119034	0.509898	-0.213019	0.722917
Correcta	left ankle (Y)	0.383212	0.341366	0.803305	0	0.803305
Correcta	right ankle (X)	0.0869236	0.110191	0.479543	-0.248161	0.727704
Correcta	right ankle (Y)	0.397276	0.3537	0.833136	0	0.833136

Correcta	left heel (X)	0.086493	0.110905	0.493178	-0.275322	0.7685
Correcta	left heel (Y)	0.375092	0.334557	0.809874	0	0.809874
Correcta	right heel (X)	0.0701478	0.101837	0.457944	-0.307234	0.765179
Correcta	right heel (Y)	0.389697	0.347332	0.840412	0	0.840412
Correcta	left foot index (X)	0.102267	0.121108	0.507368	-0.286099	0.793467
Correcta	left foot index (Y)	0.437894	0.389486	0.922219	0	0.922219
Correcta	right foot index (X)	0.0894583	0.114052	0.484155	-0.306313	0.790468
Correcta	right foot index (Y)	0.454271	0.403721	0.947087	0	0.947087
Incorrecta	nose (X)	0.518791	0.358167	0.95979	0	0.95979
Incorrecta	nose (Y)	0.341978	0.261214	0.889328	0	0.889328
Incorrecta	left eye (inner) (X)	0.527147	0.363445	0.962623	0	0.962623
Incorrecta	left eye (inner) (Y)	0.333271	0.257025	0.875904	0	0.875904
Incorrecta	left eye (X)	0.527195	0.363435	0.961586	0	0.961586
Incorrecta	left eye (Y)	0.331945	0.256196	0.874291	0	0.874291
Incorrecta	left eye (outer) (X)	0.527227	0.36341	0.960478	0	0.960478
Incorrecta	left eye (outer) (Y)	0.330386	0.255269	0.872852	0	0.872852
Incorrecta	right eye (inner) (X)	0.527291	0.363511	0.962972	0	0.962972
Incorrecta	right eye (inner) (Y)	0.330641	0.255716	0.872657	0	0.872657
Incorrecta	right eye (X)	0.527428	0.363544	0.96196	0	0.96196
Incorrecta	right eye (Y)	0.327566	0.254028	0.868243	0	0.868243
Incorrecta	right eye (outer) (X)	0.527538	0.363559	0.960808	0	0.960808
Incorrecta	right eye (outer) (Y)	0.324123	0.252189	0.863374	0	0.863374
Incorrecta	left ear (X)	0.523019	0.360104	0.943995	0	0.943995
Incorrecta	left ear (Y)	0.309819	0.243082	0.847379	0	0.847379
Incorrecta	right ear (X)	0.523549	0.360472	0.945362	0	0.945362
Incorrecta	right ear (Y)	0.302365	0.23957	0.832473	0	0.832473
Incorrecta	mouth (left) (X)	0.509777	0.351926	0.940418	0	0.940418
Incorrecta	mouth (left) (Y)	0.336949	0.2571	0.87444	0	0.87444
Incorrecta	mouth (right) (X)	0.509798	0.35194	0.942366	0	0.942366
Incorrecta	mouth (right) (Y)	0.33361	0.255512	0.869643	0	0.869643

Incorrecta	left shoulder (X)	0.465353	0.320624	0.835964	0	0.835964
Incorrecta	left shoulder (Y)	0.286181	0.217513	0.726167	0	0.726167
Incorrecta	right shoulder (X)	0.475773	0.328374	0.866232	0	0.866232
Incorrecta	right shoulder (Y)	0.280094	0.224571	0.779083	0	0.779083
Incorrecta	left elbow (X)	0.432473	0.298238	0.842494	0	0.842494
Incorrecta	left elbow (Y)	0.368668	0.258272	0.728565	0	0.728565
Incorrecta	right elbow (X)	0.446044	0.309257	0.845795	0	0.845795
Incorrecta	right elbow (Y)	0.371219	0.265487	0.755065	0	0.755065
Incorrecta	left wrist (X)	0.443845	0.306429	0.914423	0	0.914423
Incorrecta	left wrist (Y)	0.473323	0.326988	0.857043	0	0.857043
Incorrecta	right wrist (X)	0.461595	0.320854	0.868141	0	0.868141
Incorrecta	right wrist (Y)	0.50121	0.350733	0.911868	0	0.911868
Incorrecta	left pinky (X)	0.455845	0.314441	0.938165	0	0.938165
Incorrecta	left pinky (Y)	0.480809	0.331814	0.87893	0	0.87893
Incorrecta	right pinky (X)	0.47606	0.330521	0.886062	0	0.886062
Incorrecta	right pinky (Y)	0.516954	0.360728	0.942883	0	0.942883
Incorrecta	left index (X)	0.459798	0.316953	0.933893	0	0.933893
Incorrecta	left index (Y)	0.483208	0.333692	0.879287	0	0.879287
Incorrecta	right index (X)	0.479254	0.332294	0.88062	0	0.88062
Incorrecta	right index (Y)	0.510538	0.356042	0.93282	0	0.93282
Incorrecta	left thumb (X)	0.454879	0.313702	0.922932	0	0.922932
Incorrecta	left thumb (Y)	0.482372	0.333209	0.873993	0	0.873993
Incorrecta	right thumb (X)	0.473269	0.328257	0.871609	0	0.871609
Incorrecta	right thumb (Y)	0.505388	0.352614	0.922548	0	0.922548
Incorrecta	left hip (X)	0.327395	0.229114	0.720667	0	0.720667
Incorrecta	left hip (Y)	0.344904	0.249665	0.769783	0	0.769783
Incorrecta	right hip (X)	0.324037	0.227654	0.699902	0	0.699902
Incorrecta	right hip (Y)	0.348344	0.257436	0.815422	0	0.815422
Incorrecta	left knee (X)	0.217943	0.161042	0.71432	0	0.71432
Incorrecta	left knee (Y)	0.398526	0.279073	0.787304	0	0.787304

Incorrecta	right knee (X)	0.208281	0.155828	0.649307	0	0.649307
Incorrecta	right knee (Y)	0.406959	0.286407	0.829672	0	0.829672
Incorrecta	left ankle (X)	0.105541	0.104834	0.784074	-0.0875045	0.871579
Incorrecta	left ankle (Y)	0.405973	0.282072	0.762507	0	0.762507
Incorrecta	right ankle (X)	0.0904802	0.0994103	0.699641	-0.128433	0.828074
Incorrecta	right ankle (Y)	0.41397	0.288234	0.777809	0	0.777809
Incorrecta	left heel (X)	0.085833	0.0981889	0.798285	-0.148964	0.947249
Incorrecta	left heel (Y)	0.391071	0.272929	0.775602	0	0.775602
Incorrecta	right heel (X)	0.070317	0.0933103	0.719326	-0.175305	0.89463
Incorrecta	right heel (Y)	0.396805	0.278396	0.798776	0	0.798776
Incorrecta	left foot index (X)	0.0973982	0.106996	0.811952	-0.155366	0.967318
Incorrecta	left foot index (Y)	0.473425	0.326837	0.861378	0	0.861378
Incorrecta	right foot index (X)	0.0825948	0.101909	0.724273	-0.179043	0.903317
Incorrecta	right foot index (Y)	0.484878	0.335587	0.869946	0	0.869946
Global	nose (X)	0.474387	0.372822	0.95979	0	0.95979
Global	nose (Y)	0.330929	0.284352	0.923014	0	0.923014
Global	left eye (inner) (X)	0.481611	0.378149	0.962623	0	0.962623
Global	left eye (inner) (Y)	0.320293	0.277611	0.920705	0	0.920705
Global	left eye (X)	0.481617	0.37812	0.961586	0	0.961586
Global	left eye (Y)	0.318809	0.276472	0.918398	0	0.918398
Global	left eye (outer) (X)	0.481611	0.378077	0.960478	0	0.960478
Global	left eye (outer) (Y)	0.317086	0.275181	0.915451	0	0.915451
Global	right eye (inner) (X)	0.48139	0.377987	0.962972	0	0.962972
Global	right eye (inner) (Y)	0.318423	0.276637	0.917519	0	0.917519
Global	right eye (X)	0.481223	0.377839	0.96196	0	0.96196
Global	right eye (Y)	0.31575	0.274914	0.913205	0	0.913205
Global	right eye (outer) (X)	0.481011	0.377658	0.960808	0	0.960808
Global	right eye (outer) (Y)	0.312739	0.273002	0.908472	0	0.908472
Global	left ear (X)	0.476088	0.37357	0.943995	0	0.943995
Global	left ear (Y)	0.298142	0.261546	0.875618	0	0.875618

Global	right ear (X)	0.475328	0.373143	0.945362	0	0.945362
Global	right ear (Y)	0.293513	0.259668	0.868351	0	0.868351
Global	mouth (left) (X)	0.465595	0.365966	0.940418	0	0.940418
Global	mouth (left) (Y)	0.328238	0.281375	0.903963	0	0.903963
Global	mouth (right) (X)	0.465198	0.365701	0.942366	0	0.942366
Global	mouth (right) (Y)	0.325626	0.279989	0.898981	0	0.898981
Global	left shoulder (X)	0.422515	0.331886	0.847306	0	0.847306
Global	left shoulder (Y)	0.28	0.238026	0.751023	0	0.751023
Global	right shoulder (X)	0.427059	0.337092	0.866232	0	0.866232
Global	right shoulder (Y)	0.286185	0.251416	0.827523	0	0.827523
Global	left elbow (X)	0.391077	0.307371	0.842494	0	0.842494
Global	left elbow (Y)	0.353485	0.280942	0.798809	0	0.798809
Global	right elbow (X)	0.395091	0.314134	0.845795	0	0.845795
Global	right elbow (Y)	0.371692	0.301035	0.92978	0	0.92978
Global	left wrist (X)	0.400825	0.315057	0.914423	0	0.914423
Global	left wrist (Y)	0.450205	0.355029	0.935731	0	0.935731
Global	right wrist (X)	0.408233	0.324037	0.868141	0	0.868141
Global	right wrist (Y)	0.492425	0.392973	1.08222	0	1.08222
Global	left pinky (X)	0.412558	0.323999	0.938165	0	0.938165
Global	left pinky (Y)	0.456398	0.359895	0.972936	0	0.972936
Global	right pinky (X)	0.422314	0.334606	0.886062	0	0.886062
Global	right pinky (Y)	0.507434	0.404902	1.15679	0	1.15679
Global	left index (X)	0.415562	0.326246	0.933893	0	0.933893
Global	left index (Y)	0.459461	0.362464	0.982433	0	0.982433
Global	right index (X)	0.425878	0.336907	0.88062	0	0.88062
Global	right index (Y)	0.500401	0.399013	1.12223	0	1.12223
Global	left thumb (X)	0.410787	0.32263	0.922932	0	0.922932
Global	left thumb (Y)	0.459312	0.362347	0.982168	0	0.982168
Global	right thumb (X)	0.420351	0.332636	0.871609	0	0.871609
Global	right thumb (Y)	0.495035	0.39454	1.10793	0	1.10793

Global	left hip (X)	0.295802	0.236302	0.720667	0	0.720667
Global	left hip (Y)	0.333688	0.271471	0.773461	0	0.773461
Global	right hip (X)	0.291589	0.234252	0.702599	0	0.702599
Global	right hip (Y)	0.342157	0.281736	0.843163	0	0.843163
Global	left knee (X)	0.199945	0.167904	0.71432	0	0.71432
Global	left knee (Y)	0.3828	0.304375	0.799709	0	0.799709
Global	right knee (X)	0.190364	0.162092	0.649307	0	0.649307
Global	right knee (Y)	0.393865	0.314497	0.84561	0	0.84561
Global	left ankle (X)	0.104095	0.112168	0.784074	-0.213019	0.997093
Global	left ankle (Y)	0.394593	0.313332	0.803305	0	0.803305
Global	right ankle (X)	0.0887019	0.104954	0.699641	-0.248161	0.947802
Global	right ankle (Y)	0.405623	0.32274	0.833136	0	0.833136
Global	left heel (X)	0.086163	0.104741	0.798285	-0.275322	1.07361
Global	left heel (Y)	0.383081	0.305407	0.809874	0	0.809874
Global	right heel (X)	0.0702324	0.0976668	0.719326	-0.307234	1.02656
Global	right heel (Y)	0.393251	0.314777	0.840412	0	0.840412
Global	left foot index (X)	0.0998323	0.114296	0.811952	-0.286099	1.09805
Global	left foot index (Y)	0.45566	0.359968	0.922219	0	0.922219
Global	right foot index (X)	0.0860266	0.108206	0.724273	-0.306313	1.03059
Global	right foot index (Y)	0.469575	0.371536	0.947087	0	0.947087

Con estos resultados podemos ver la ubicación media de diversas partes del cuerpo, por ejemplo la nariz está a una altura de 0.533876 del eje X, mientras que el pie derecho está a una altura de 0.268688 respecto al eje X. Las desviaciones nos sirven para ver cuanto varía la posición del landmark respecto a su media, y como podemos observar, obtenemos valores normales teniendo en cuenta que el movimiento es una flexión. Si nos vamos a partes más fijas a lo largo de una flexión, como pueden ser los pies que permanecen inmóviles, obtenemos desviaciones mucho más bajas en el eje X, como 0.0780976 en el pie izquierdo y 0.0738948 en el derecho.

Ángulos que forman los landmarks

Unas características muy importantes a la hora de determinar si una flexión se realiza correctamente son los ángulos articulares (por ejemplo, doblar las rodillas es síntoma de que la técnica no es buena).

In [22]:
```
KEYPOINTS = {
    'nariz': 0,
    'hombro_I': 11, 'hombro_D': 12,
    'codo_I': 13,   'codo_D': 14,
    'muñeca_I': 15, 'muñeca_D': 16,
    'cadera_I': 23, 'cadera_D': 24,
    'rodilla_I': 25, 'rodilla_D': 26,
    'tobillo_I': 27, 'tobillo_D': 28
}

def get_coords(landmarks, idx):
    """
    Devuelve las coordenadas del landmark idx.
    Funciona con landmarks en formato:
       2D → [x,y]
       3D → [x,y,z]
       4D → [x,y,z,visibility]
    """
    dims = landmarks.shape[-1] // 33
    start = idx * dims
    end = start + dims
    return landmarks[start:end]

def calculate_angle(p1, p2_vertex, p3):
    """
    Calcula el ángulo en B dado A-B-C.
    Robusto frente:
    - vectores cero
    - NaNs
    - rango del coseno
    """

    if np.any(np.isnan([p1, p2_vertex, p3])):
        return 0.0

    # Calculamos los vectores
    vector_ba = p1 - p2_vertex
    vector_bc = p3 - p2_vertex

    norm_ba = np.linalg.norm(vector_ba)
    norm_bc = np.linalg.norm(vector_bc)

    # Manejamos el caso de vectores de longitud cero
    if norm_ba < 1e-6 or norm_bc < 1e-6:
        return 0.0

    # Calculamos el coseno del ángulo usando la fórmula del producto punto.
    cos_theta = np.dot(vector_ba, vector_bc) / (norm_ba * norm_bc + 1e-6) # Se a

    # Nos aseguramos que el valor esté en el rango [-1, 1] para evitar errores n
    cos_theta = np.clip(cos_theta, -1.0, 1.0)

    return np.degrees(np.arccos(cos_theta))

def get_key_joint_angles(frame):
    """
    frame: vector 1D con landmarks (66, 99 o 132 valores)

    Devuelve un dict con los ángulos claves.
```

```
    """
    angles = {}

    # Para abreviar
    get = lambda name: get_coords(frame, KEYPOINTS[name])

    # Ángulos de codo
    angles['codo_I'] = calculate_angle(get('hombro_I'), get('codo_I'), get('muñe
    angles['codo_D'] = calculate_angle(get('hombro_D'), get('codo_D'), get('muñe

    # Alineación del tronco
    angles['cadera_I_tronco'] = calculate_angle(get('hombro_I'), get('cadera_I')
    angles['cadera_D_tronco'] = calculate_angle(get('hombro_D'), get('cadera_D')

    # Rodillas (sentadillas)
    angles['rodilla_I'] = calculate_angle(get('cadera_I'), get('rodilla_I'), get
    angles['rodilla_D'] = calculate_angle(get('cadera_D'), get('rodilla_D'), get

    # Cadera (sentadillas) - reutilizamos el cálculo
    angles['cadera_I_squat'] = angles['cadera_I_tronco']
    angles['cadera_D_squat'] = angles['cadera_D_tronco']

    # Tronco usando la nariz
    angles['tronco_nariz_I'] = calculate_angle(get('nariz'), get('hombro_I'), ge
    angles['tronco_nariz_D'] = calculate_angle(get('nariz'), get('hombro_D'), ge

    return angles

def landmarks_to_angle_df(dataset, add_video_id=False):
    """
    dataset: array con forma (n_videos, n_frames, dims)
    dims = 66, 99 o 132 según tengas 33*2, 33*3, 33*4 valores.

    Devuelve:
        DataFrame con 8 columnas de ángulos.
        (optional) columnas video_id y frame_id.
    """

    n_videos, n_frames, dims = dataset.shape

    rows = []

    # Recorremos todos los vídeos y frames
    for video_idx in range(n_videos):
        for frame_idx in range(n_frames):

            # Extraemos el vector de landmarks 1D para el frame actual.
            frame = dataset[video_idx, frame_idx]

            # Si la suma de todas las coordenadas es cero, se asume que es un fr
            if frame.sum() == 0:
                continue

            # Llamamos a la función ya definida para calcular los ángulos clave
            angles = get_key_joint_angles(frame)

            # Añadimos IDs si se solicita. Por trazabilidad.
            if add_video_id:
                angles['video_id'] = video_idx
                angles['frame_id'] = frame_idx
```

```
            # Añadimos el diccionario de ángulos (e indices) a la lista de filas
            rows.append(angles)

    # Convertimos la lista de diccionarios (rows) en un DataFrame de Pandas dond
    return pd.DataFrame(rows)
```

In [23]:
```
# Vamos a almacenar en discos los arrays con los ángulos articulares de cada fra
def calcular_angulos_esqueleto(landmarks):
    """
    landmarks: array 1D con forma (66,)

    Devuelve un array 1D con los ángulos clave del esqueleto.
    """

    angles_dict = get_key_joint_angles(landmarks)
    return np.array(list(angles_dict.values()))

angulos_procesados_correct_pu = np.apply_along_axis(
    func1d=calcular_angulos_esqueleto,
    axis=2, #-1
    arr=correct_push_ups
)

angulos_procesados_wrong_pu = np.apply_along_axis(
    func1d=calcular_angulos_esqueleto,
    axis=2, #-1
    arr=wrong_push_ups
)

# Almacenamos en discos los arrays con los ángulos articulares procesados
np.save("../data/processed/correct_push_ups_angles.npy", angulos_procesados_corr
np.save("../data/processed/wrong_push_ups_angles.npy",   angulos_procesados_wron
```

In [24]:
```
# Almacenamos en discos los arrays con los ángulos articulares procesados
np.save("../data/processed/correct_push_ups_angles.npy", angulos_procesados_corr
np.save("../data/processed/wrong_push_ups_angles.npy",   angulos_procesados_wron
```

In [25]:
```
np.array(list(get_key_joint_angles(seq[0]).values()))
```

Out[25]:
```
array([178.78214346, 178.63288231, 163.76861214, 163.1303037 ,
       176.36427452, 172.52492135, 163.76861214, 163.1303037 ,
       145.12399083, 173.38601013])
```

In [26]:
```
angles_correct = landmarks_to_angle_df(correct_push_ups_escaladas, add_video_id=
angles_wrong = landmarks_to_angle_df(incorrect_push_ups_escaladas, add_video_id=
```

In [27]:
```
# Añadimos etiquetas
angles_correct["Clase"] = "Correcto"
angles_wrong["Clase"] = "Incorrecto"

# Unimos las dos tablas
df_combined = pd.concat([angles_correct, angles_wrong], ignore_index=True)
```

In [28]:
```
print(df_combined.shape) # tiene 9979 filas y no 16000 (100*160) porque se han e
df_combined.head()
```

```
(9979, 13)
```

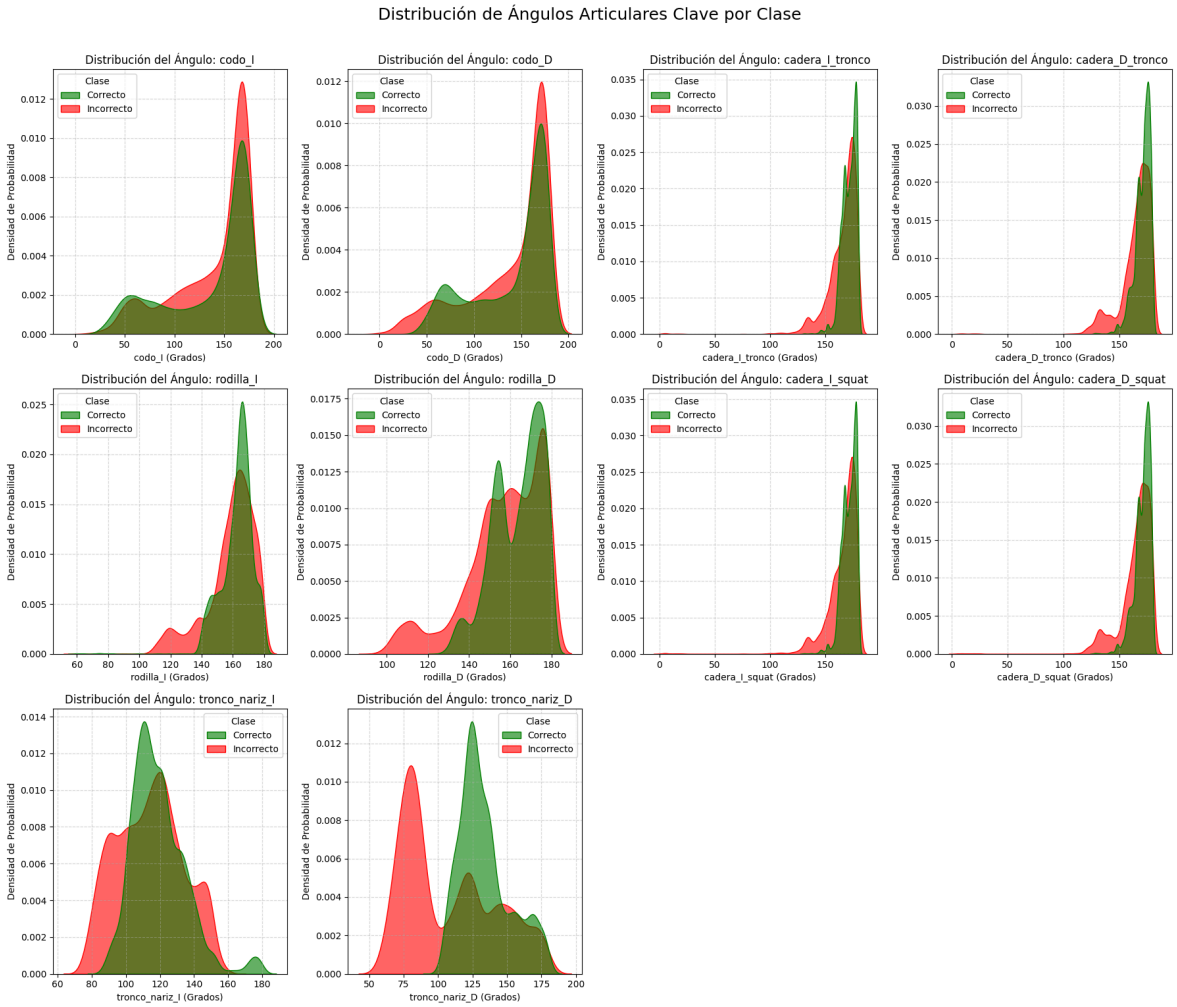
Out[28]:

	codo_I	codo_D	cadera_I_tronco	cadera_D_tronco	rodilla_I	rodilla_D	ca
0	169.647655	158.547336	171.964876	171.393471	159.780545	152.909449	
1	167.251643	158.140467	174.992269	173.216054	162.065725	151.476411	
2	165.870691	154.897574	177.878500	174.211190	164.248656	151.768960	
3	162.836663	149.889841	179.561310	173.020830	165.001736	150.878770	
4	159.006911	143.900044	178.532774	170.431589	165.254383	151.501216	

In [29]:

```
# Definimos las columnas de ángulos a graficar excluyendo las columnas que no so
columns_to_exclude = ['Clase', 'video_id', 'frame_id']
all_angle_columns = [col for col in df_combined.columns if col not in columns_to
num_plots = len(all_angle_columns)

# Configuramos el lienzo
cols = 4
rows = int(np.ceil(num_plots / cols))

fig, axes = plt.subplots(
    nrows=rows,
    ncols=min(num_plots, cols),
    figsize=(18, 5 * rows)
)
# Aplanamos los ejes para facilitar el indexado
axes = axes.flatten()

# Título general
fig.suptitle('Distribución de Ángulos Articulares Clave por Clase',
             fontsize=18,
             y=1.02)

# Bucle para generar los gráficos de densidad KDE
for i, angle_col in enumerate(all_angle_columns):

    # usamos seaborn.kdeplot para comparar las distribuciones
    sns.kdeplot(
        data=df_combined,
        x=angle_col,
        hue='Clase', # Diferencia las clases por color
        fill=True, # Rellena el área bajo la curva
        palette={'Correcto': 'green', 'Incorrecto': 'red'},
        alpha=0.6,
        ax=axes[i]
    )

    # Configuración de los ejes y el título
    axes[i].set_title(f'Distribución del Ángulo: {angle_col}')
    axes[i].set_xlabel(f'{angle_col} (Grados)')
    axes[i].set_ylabel('Densidad de Probabilidad')
    axes[i].grid(True, linestyle='--', alpha=0.5)

# Limpiamos subplots vacíos (solo tenemos 10 gráficos y el grid es de 12)
for empty_idx in range(num_plots, rows * cols):
    if empty_idx < len(axes): # Pequeña comprobación
        fig.delaxes(axes[empty_idx])

# Ajustamos diseño y mostramos
```

```
plt.tight_layout(rect=[0, 0, 1, 1.01])
plt.show()
```



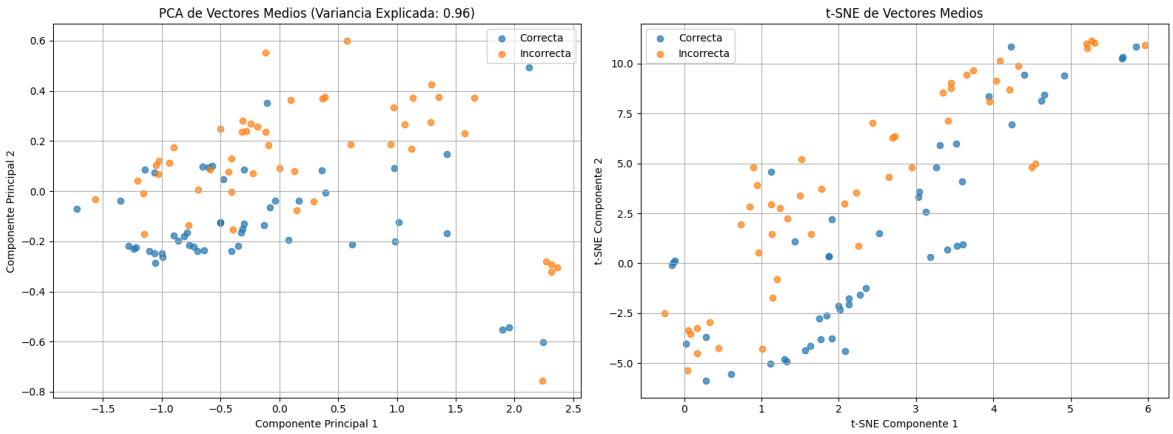
Parece que las distribuciones de los ángulos articulares de cada clase son muy similares. Si nos fijamos en las distribuciones de los ángulos de las rodillas, por debajo de 150º encontramos más frames de posturas incorrectas que correctas (cuanto menor sea el ángulo más se dobla la articulación).

Reducción de dimensionalidad

Aplicamos PCA o t-SNE sobre los vectores medios de cada vídeo → ver si las clases se separan visualmente.

In [30]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

# Calculamos el vector medio (pose promedio) por vídeo
mean_vectors_correct = np.mean(correct_push_ups_escaladas, axis=1) # Media a lo
mean_vectors_incorrect = np.mean(incorrect_push_ups_escaladas, axis=1)

# Combinamos los vectores medios en una sola matriz (X_mean) y creamos etiquetas
X_mean = np.vstack((mean_vectors_correct, mean_vectors_incorrect))
y = np.array(['Correcta'] * mean_vectors_correct.shape[0] + ['Incorrecta'] * mea

# Aplicamos PCA
```

```
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_mean)
explained_variance = pca.explained_variance_ratio_.sum()

# Aplicamos t-SNE
tsne = TSNE(n_components=2, random_state=42, init='pca', learning_rate='auto')
X_tsne = tsne.fit_transform(X_mean)

# Generamos gráficos
plt.figure(figsize=(16, 6))

# Subplot 1: PCA
plt.subplot(1, 2, 1)
for label in np.unique(y):
    subset = X_pca[y == label]
    plt.scatter(subset[:, 0], subset[:, 1], label=label, alpha=0.7)

plt.title(f'PCA de Vectores Medios (Variancia Explicada: {explained_variance:.2f
plt.xlabel('Componente Principal 1')
plt.ylabel('Componente Principal 2')
plt.legend()
plt.grid(True)

# Subplot 2: t-SNE
plt.subplot(1, 2, 2)
for label in np.unique(y):
    subset = X_tsne[y == label]
    plt.scatter(subset[:, 0], subset[:, 1], label=label, alpha=0.7)

plt.title('t-SNE de Vectores Medios')
plt.xlabel('t-SNE Componente 1')
plt.ylabel('t-SNE Componente 2')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig('visualizacion_separacion_clases.png')
plt.show()
```



En el PCA, a pesar de que ambos conjuntos no estén completamente separados (se mezclan en el centro) podemos ver cómo los puntos naranjas (flexiones incorrectas) se agrupan en la parte superior derecha (valores más altos), mientras que los puntos que hacen referencia a las flexiones bien hechas, se agrupan en la parte media-inferior izquierda (valores más bajos)

El gráfico de t-SNE confirma la separación encontrada por PCA. t-SNE muestra una clara estructura lineal donde los puntos de una clase siguen una línea que está por encima o por debajo de los puntos de la otra clase, con una separación mínima. La mayoría de los puntos incorrectos (naranja) están desplazados ligeramente por encima y a la izquierda de los puntos correctos (azul). Esto sugiere que la diferencia entre las clases es ciertamente consistente.
Concluyendo, mediante t-SNE observamos que en el espacio de alta dimensión (los 66 landmarks), los videos de una clase están íntimamente relacionados entre sí, y que la distancia entre un video correcto y un video incorrecto es grande.

Modelo Baseline

Se implementará un pipeline inicial simple que incluirá una pila de 1D-CNN, seguida capas densas con activación sigmoide para la clasificación binaria.

In [48]:
```
import numpy as np
from sklearn.model_selection import train_test_split, GroupKFold, KFold
import torch
```

In [ ]:
```
# Cargamos los arrays con los landmarks
X_correct_original = np.load('../Data/Processed/correct_push_ups_landmarks.npy')
X_incorrect_original = np.load('../Data/Processed/wrong_push_ups_landmarks.npy')

X_original = np.vstack((X_correct_original, X_incorrect_original))
y_original = np.array([1]*X_correct_original.shape[0] + [0]*X_incorrect_original

# Convertimos a tensores de PyTorch
X_tensor_original = torch.tensor(X_original, dtype=torch.float32)
y_tensor_original = torch.tensor(y_original, dtype=torch.long) # Las etiquetas a

# Obtenemos el número de muestras (N)
n_samples = X_tensor_original.shape[0]

# Creamos un array con los índices de los grupos (videos originales)
original_index = np.arange(n_samples)
```

In [ ]:
```
import torch
import torch.nn as nn

input_channels = X_tensor_original.shape[2]  # Número de características por fra
seq_len = X_tensor_original.shape[1]         # Número de frames por secuencia (v

class Baseline_CNN1D(nn.Module):
    def __init__(self, num_classes, input_channels=66, seq_len=50):
        super().__init__()

        # Cada coordenada se procesa de forma independiente

        self.features = nn.Sequential(
            nn.Conv1d(in_channels=input_channels, out_channels=input_channels, k
            nn.BatchNorm1d(input_channels),
            nn.ReLU(),
            nn.Dropout(0.1),
```

```
            nn.Conv1d(in_channels=input_channels, out_channels=input_channels, k
            nn.BatchNorm1d(input_channels),
            nn.ReLU(),

            nn.MaxPool1d(kernel_size=2)

        )

        # Cálculo de las dimensiones para el aplanado
        flatten_dim = input_channels * (seq_len // 2)


        # Capas Densas (FFN) - Se combina la información de todas las coordenada
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(flatten_dim, 64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, num_classes)
        )


    def forward(self, x):
        # x original: (batch_size, seq_len, input_channels) -> (N, 160, 66)

        x = x.permute(0, 2, 1)  # (N, input_channels, seq_len)

        # Extracción de características (cada canal por separado)
        x = self.features(x)

        # Clasificación (combinando la información de todos los canales)
        x = self.classifier(x)

        return x
```

In [ ]:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import GroupKFold
import numpy as np
import copy

# FUNCIÓN DE ENTRENAMIENTO (UNA ÉPOCA)
def train_one_epoch(model, loader, criterion, optimizer, device):
    """Ejecuta el paso de entrenamiento para una sola época."""
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for X_batch, y_batch in loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
```

```
        running_loss += loss.item() * X_batch.size(0)
        _, predicted = torch.max(outputs, 1)
        correct += (predicted == y_batch).sum().item()
        total += y_batch.size(0)

    avg_loss = running_loss / len(loader.dataset)
    avg_acc = correct / total
    return avg_loss, avg_acc

# FUNCIÓN DE VALIDACIÓN (UNA ÉPOCA)
def validate_one_epoch(model, loader, criterion, device):
    """Ejecuta la evaluación sin calcular gradientes."""
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0

    # Listas para guardar predicciones (útil para el paso final del fold)
    preds_list = []
    labels_list = []

    with torch.no_grad():
        for X_batch, y_batch in loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)

            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)

            running_loss += loss.item() * X_batch.size(0)
            _, predicted = torch.max(outputs, 1)
            correct += (predicted == y_batch).sum().item()
            total += y_batch.size(0)

            # Guardamos predicciones por si es la última pasada
            preds_list.extend(predicted.cpu().numpy())
            labels_list.extend(y_batch.cpu().numpy())

    avg_loss = running_loss / len(loader.dataset)
    avg_acc = correct / total

    return avg_loss, avg_acc, preds_list, labels_list


# ENTRENAMIENTO DE UN SOLO FOLD
def train_single_fold(model, train_loader, val_loader, epochs, lr, patience, dev
    """Entrena un modelo desde cero para un fold específico con Early Stopping."

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    history = {'loss': [], 'acc': [], 'val_loss': [], 'val_acc': []}

    # Variables de Early Stopping
    best_val_loss = float('inf')
    patience_counter = 0
    best_weights = None

    for epoch in range(epochs):
        # Entrenamos
```

```
        train_loss, train_acc = train_one_epoch(model, train_loader, criterion,

        # Validamos
        val_loss, val_acc, _, _ = validate_one_epoch(model, val_loader, criterio

        # Guardamos Historia
        history['loss'].append(train_loss)
        history['acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)

        # Lógica Early Stopping
        if patience is not None:
            if val_loss < best_val_loss:
                best_val_loss = val_loss
                patience_counter = 0
                best_weights = copy.deepcopy(model.state_dict())
            else:
                patience_counter += 1
                if patience_counter >= patience:
                    print(f"    ⏹️ Early stopping en época {epoch+1}")
                    break

    # Restauramos el mejor modelo y hacer inferencia final
    if best_weights:
        model.load_state_dict(best_weights)

    # Hacemos una última pasada de validación con el MEJOR modelo para sacar las
    final_val_loss, final_val_acc, final_preds, final_labels = validate_one_epoc

    return final_val_acc, history, final_preds, final_labels


# 4. FUNCIÓN PRINCIPAL (EJECUCIÓN DE UN EXPERIMENTO K-FOLD)
def run_kfold_experiment_pytorch(model_class, model_params, X_full, y_full, grou
                                 epochs, batch_size, lr=0.001, n_splits=5,

    # 1. Asegurar tipos numpy para que sklearn funcione
    if isinstance(X_full, torch.Tensor): X_full = X_full.cpu().numpy()
    if isinstance(y_full, torch.Tensor): y_full = y_full.cpu().numpy()

    # 2. Configurar GroupKFold
    # IMPORTANTE: GroupKFold no acepta shuffle=True ni random_state.
    # La aleatoriedad depende de cómo estén ordenados los grupos en tu array 'gr
    gkf = GroupKFold(n_splits=n_splits)

    # Acumuladores Globales
    fold_accuracies = []
    fold_histories = []

    # Listas TEMPORALES para guardar resultados desordenados
    global_y_true_unsorted = []
    global_y_pred_unsorted = []
    global_indices = [] # <--- NECESARIO PARA REORDENAR LUEGO

    print(f"🚀 Iniciando GroupKFold Modular ({n_splits} splits) en {device}..."
    print(f"   Protección contra Data Leakage activa (Grupos únicos detectados:
    print("===" * 20)

    # 3. Bucle K-Fold pasando 'groups'
```

```
    for fold_idx, (train_index, val_index) in enumerate(gkf.split(X_full, y_full
        print(f"\n🔄 Pliegue {fold_idx + 1}/{n_splits}")

        # A. Preparar Tensores y Loaders
        X_train_t = torch.tensor(X_full[train_index], dtype=torch.float32)
        y_train_t = torch.tensor(y_full[train_index], dtype=torch.long)
        X_val_t = torch.tensor(X_full[val_index], dtype=torch.float32)
        y_val_t = torch.tensor(y_full[val_index], dtype=torch.long)

        train_loader = DataLoader(TensorDataset(X_train_t, y_train_t), batch_siz
        val_loader = DataLoader(TensorDataset(X_val_t, y_val_t), batch_size=batc

        # B. Inicializar Modelo (Nuevo para cada fold)
        model = model_class(**model_params).to(device)

        # C. Delegar el trabajo a la función 'train_single_fold'
        val_acc, history, fold_preds, fold_labels = train_single_fold(
            model, train_loader, val_loader, epochs, lr, patience, device
        )

        # D. Recolectar resultados
        print(f"    ✅ Fin Fold {fold_idx+1}. Mejor Acc Val: {val_acc:.4f}")

        fold_accuracies.append(val_acc)
        fold_histories.append(history)

        # Guardamos datos crudos Y LOS ÍNDICES de este fold
        global_y_true_unsorted.extend(fold_labels)
        global_y_pred_unsorted.extend(fold_preds)
        global_indices.extend(val_index) # <--- CLAVE PARA EL ORDEN


    # 4. REORDENAMIENTO FINAL (CRÍTICO)
    # GroupKFold procesa los bloques en desorden. Si no hacemos esto,
    # y_pred[0] NO corresponderá a X_full[0].

    global_indices_np = np.array(global_indices)
    sorted_order = np.argsort(global_indices_np) # Obtenemos los índices que ora

    # Reordenamos las predicciones y etiquetas usando ese orden
    final_y_true = np.array(global_y_true_unsorted)[sorted_order]
    final_y_pred = np.array(global_y_pred_unsorted)[sorted_order]

    # Resultados Finales
    mean_acc = np.mean(fold_accuracies)
    std_acc = np.std(fold_accuracies)

    print("===" * 20)
    print(f"📊 Resultados GroupKFold:")
    print(f"| Accuracy Medio: {mean_acc:.4f} (+/- {std_acc:.4f})")

    return mean_acc, std_acc, fold_histories, final_y_true, final_y_pred
```

EXP-CNN-01

In [ ]:
```
# 1. Definimos los parámetros
params_baseline = {
    'num_classes': 2,
```

```
    'input_channels': input_channels,
    'seq_len': seq_len
}

# 2. Entrenamos usando la función run_kfold_experiment_pytorch
mean_acc, std_acc, histories, y_true, y_pred = run_kfold_experiment_pytorch(
    model_class=Baseline_CNN1D,
    model_params=params_baseline,
    X_full=X_tensor_original,
    y_full=y_tensor_original,
    groups=original_index,
    epochs=100,
    batch_size=32,
    n_splits=5,
    patience=35,
    device='cpu'
)
```

```
🚀 Iniciando GroupKFold Modular (5 splits) en cpu...
   Protección contra Data Leakage activa (Grupos únicos detectados: 100)
============================================================

🔄 Pliegue 1/5
    ⏹️ Early stopping en época 49
    ✅ Fin Fold 1. Mejor Acc Val: 0.8000

🔄 Pliegue 2/5
    ⏹️ Early stopping en época 53
    ✅ Fin Fold 2. Mejor Acc Val: 0.7000

🔄 Pliegue 3/5
    ⏹️ Early stopping en época 37
    ✅ Fin Fold 3. Mejor Acc Val: 0.5000

🔄 Pliegue 4/5
    ⏹️ Early stopping en época 50
    ✅ Fin Fold 4. Mejor Acc Val: 0.6500

🔄 Pliegue 5/5
    ⏹️ Early stopping en época 61
    ✅ Fin Fold 5. Mejor Acc Val: 0.8000
============================================================
📊 Resultados GroupKFold:
| Accuracy Medio: 0.6900 (+/- 0.1114)
```

Evaluación con Bootstrapping

In [ ]:

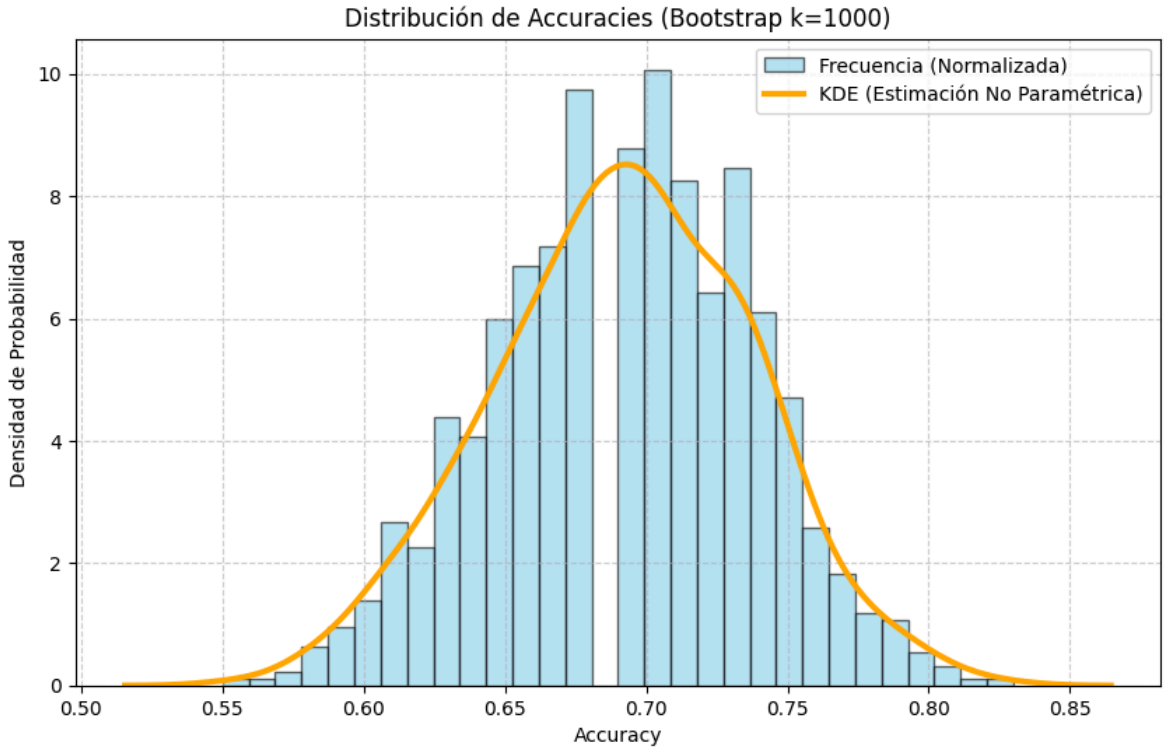
```
from sklearn.utils import resample
from sklearn.metrics import accuracy_score
import numpy as np

def calcular_bootstrap_intervalo(y_true, y_pred, n_iterations=1000, alpha=0.95):
    """
    Calcula el intervalo de confianza usando Bootstrapping sobre las prediccione
    """
    stats = []

    # Bucle de simulaciones (1000 veces)
    for i in range(n_iterations):
```

```
        # Generamos índices aleatorios con reemplazo
        # (Simulamos meter la mano en la bolsa y sacar bolas repetidas)
        indices = resample(range(len(y_pred)), replace=True)

        # Creamos el "dataset virtual"
        sample_pred = y_pred[indices]
        sample_true = y_true[indices]

        # Calculamos el accuracy de esta simulación
        score = accuracy_score(sample_true, sample_pred)
        stats.append(score)

    # Ordenamos los 1000 resultados de menor a mayor
    stats = np.sort(stats)

    # Calculamos los límites (cortamos las colas)
    lower_percentile = ((1.0 - alpha) / 2.0) * 100
    upper_percentile = (alpha + ((1.0 - alpha) / 2.0)) * 100

    lower = np.percentile(stats, lower_percentile)
    upper = np.percentile(stats, upper_percentile)

    print(f"Accuracy Promedio (Bootstrap): {np.mean(stats):.4f}")
    print(f"Intervalo de Confianza {int(alpha*100)}%: [{lower:.4f}, {upper:.4f}]

    return lower, upper, stats
```

In [54]:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Graficamos el histograma de accuracies bootstrap
n_iterations = 1000
_, _, stats = calcular_bootstrap_intervalo(y_true, y_pred, n_iterations=n_iterat


plt.figure(figsize=(10, 6))

# 1. Histograma NORMALIZADO (density=True)
plt.hist(stats, bins=30, color='skyblue', edgecolor='black',
         density=True, alpha=0.6, label='Frecuencia (Normalizada)')

# 2. Gráfico KDE (Estimación de Densidad de Kernel)
sns.kdeplot(stats, color='orange', lw=3, label='KDE (Estimación No Paramétrica)'

plt.title(f'Distribución de Accuracies (Bootstrap k={n_iterations})')
plt.xlabel('Accuracy')
plt.ylabel('Densidad de Probabilidad') # Ojo: Ya no es "Frecuencia" absoluta
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

```
Accuracy Promedio (Bootstrap): 0.6917
Intervalo de Confianza 95%: [0.6000, 0.7800]
```



In [55]:
```
# Guardamos el array en disco para análisis posterior
np.save('../results/bootstrap_accuracies_exp_cnn_01.npy', stats)
```

EXP-CNN-02

In [ ]:
```
X_angles_correct = np.load('../Data/Processed/correct_push_ups_angles.npy')
X_angles_incorrect = np.load('../Data/Processed/wrong_push_ups_angles.npy')

X_angles = np.vstack((X_angles_correct, X_angles_incorrect))
y_angles = np.array([1]*X_angles_correct.shape[0] + [0]*X_angles_incorrect.shape

# Convertimos a tensores de PyTorch
X_angles_tensor = torch.tensor(X_angles, dtype=torch.float32)
y_angles_tensor = torch.tensor(y_angles, dtype=torch.long) # Las etiquetas de cl

input_channels = X_angles_tensor.shape[2]  # Número de características por frame
seq_len = X_angles_tensor.shape[1]         # Número de frames por secuencia (via
```

In [57]:
```
input_channels
```

Out[57]:
```
10
```

In [ ]:
```
# 1. Definimos los parámetros
params_baseline = {
    'num_classes': 2,
    'input_channels': input_channels,
    'seq_len': seq_len
}

# 2. Entrenamos usando la función run_kfold_experiment_pytorch
mean_acc, std_acc, histories, y_true, y_pred = run_kfold_experiment_pytorch(
    model_class=Baseline_CNN1D,
    model_params=params_baseline,
    X_full=X_angles_tensor,
```

```
    y_full=y_angles_tensor,
    groups=original_index,
    epochs=100,
    batch_size=32,
    n_splits=5,
    patience=35,
    device='cpu'
)
```

```
🚀 Iniciando GroupKFold Modular (5 splits) en cpu...
   Protección contra Data Leakage activa (Grupos únicos detectados: 100)
============================================================

🔄 Pliegue 1/5
    ⏹️ Early stopping en época 49
    ✅ Fin Fold 1. Mejor Acc Val: 0.7000

🔄 Pliegue 2/5
    ⏹️ Early stopping en época 73
    ✅ Fin Fold 2. Mejor Acc Val: 0.8500

🔄 Pliegue 3/5
    ⏹️ Early stopping en época 36
    ✅ Fin Fold 3. Mejor Acc Val: 0.5000

🔄 Pliegue 4/5
    ⏹️ Early stopping en época 41
    ✅ Fin Fold 4. Mejor Acc Val: 0.6000

🔄 Pliegue 5/5
    ⏹️ Early stopping en época 56
    ✅ Fin Fold 5. Mejor Acc Val: 0.8500
============================================================
📊 Resultados GroupKFold:
| Accuracy Medio: 0.7000 (+/- 0.1378)
```

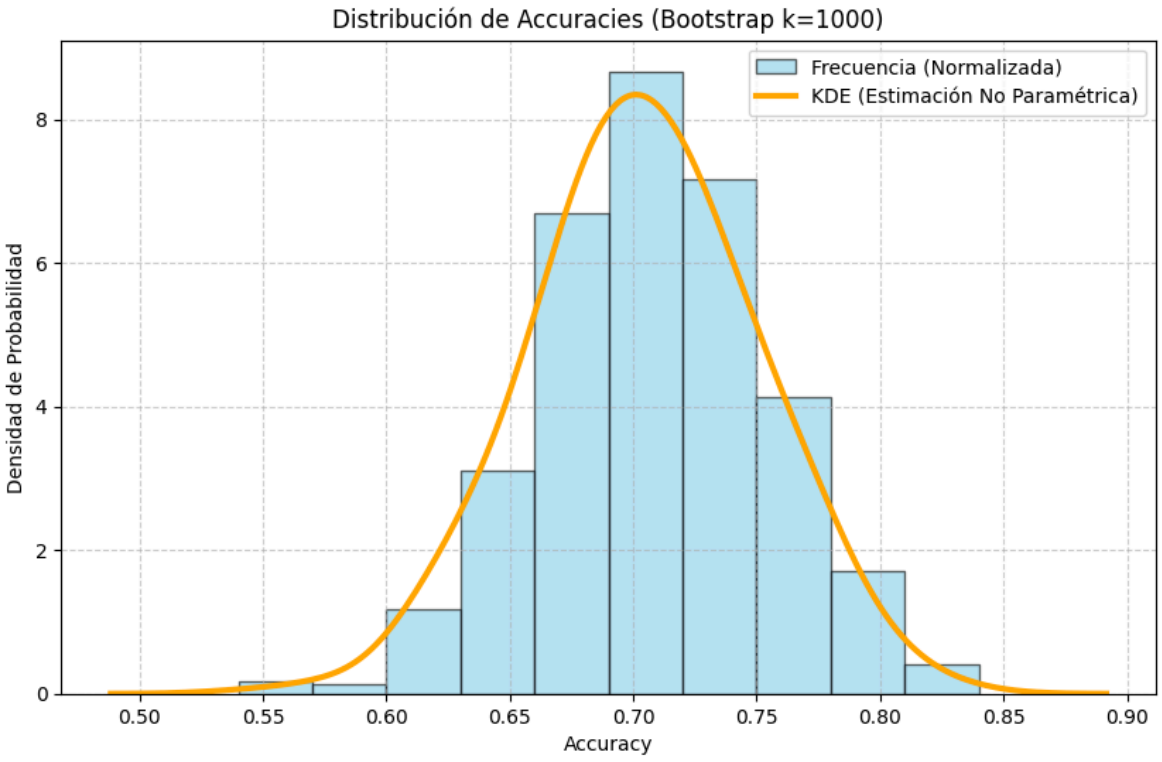
Evaluación con Bootstrapping

In [ ]:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Graficamos el histograma de accuracies bootstrap
n_iterations = 1000
_, _, stats = calcular_bootstrap_intervalo(y_true, y_pred, n_iterations=n_iterat


plt.figure(figsize=(10, 6))

# Histograma NORMALIZADO (density=True)
plt.hist(stats, bins=10, color='skyblue', edgecolor='black',
         density=True, alpha=0.6, label='Frecuencia (Normalizada)')

# Gráfico KDE (Estimación de Densidad de Kernel)
sns.kdeplot(stats, color='orange', lw=3, label='KDE (Estimación No Paramétrica)'

plt.title(f'Distribución de Accuracies (Bootstrap k={n_iterations})')
plt.xlabel('Accuracy')
plt.ylabel('Densidad de Probabilidad')
plt.legend()
```

```
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

```
Accuracy Promedio (Bootstrap): 0.7039
Intervalo de Confianza 95%: [0.6100, 0.7900]
```



In [60]:
```
# Guardamos el array en disco para análisis posterior
np.save('../results/bootstrap_accuracies_exp_cnn_02.npy', stats)
```

EXP-CNN-03

In [ ]:
```
X_correct_mirrored = np.load('../Data/Processed/mirrored_correct_push_ups_landma
X_incorrect_mirrored = np.load('../Data/Processed/mirrored_wrong_push_ups_landma

X_mirrored = np.vstack((X_correct_mirrored, X_incorrect_mirrored))
y_mirrored = np.array([1]*X_correct_mirrored.shape[0] + [0]*X_incorrect_mirrored

# Convertimos a tensores de PyTorch
X_tensor_mirrored = torch.tensor(X_mirrored, dtype=torch.float32)
y_tensor_mirrored = torch.tensor(y_mirrored, dtype=torch.long) # Las etiquetas a

input_channels = X_tensor_mirrored.shape[2]  # Número de características por fra
seq_len = X_tensor_mirrored.shape[1]         # Número de frames por secuencia (v

# Obtenemos el número de muestras (N)
n_samples_mirrored = X_tensor_mirrored.shape[0]

# Creamos un array con los índices de los grupos (videos originales)
mirrored_index = np.arange(n_samples_mirrored)
```

In [62]:
```
# Creamos un array con los índices de los grupos (videos originales)
groups = np.concatenate([original_index, mirrored_index])

# Concatenamos los datos originales y aumentados
X_tensor_concat = torch.cat((X_tensor_original, X_tensor_mirrored), dim=0)
```

```
y_tensor_concat = torch.cat((y_tensor_original, y_tensor_mirrored), dim=0)

groups
```

Out[62]:
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
       34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50,
       51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67,
       68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84,
       85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99,  0,  1,
        2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17, 18,
       19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35,
       36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52,
       53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69,
       70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86,
       87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

In [63]:
```
[fold for fold in GroupKFold(n_splits=5).split(X_tensor_concat, y_tensor_concat,
```

```
Out[63]: [(array([  0,   1,   2,   4,   5,   6,   7,   9,  10,  11,  12,  14,  15,
                   16,  17,  19,  20,  21,  22,  23,  25,  26,  27,  28,  30,  31,
                   32,  33,  35,  36,  37,  38,  40,  41,  42,  43,  45,  46,  47,
                   48,  50,  51,  52,  53,  55,  56,  57,  58,  60,  61,  62,  63,
                   65,  66,  67,  68,  70,  71,  72,  73,  75,  76,  77,  78,  80,
                   81,  82,  83,  85,  86,  87,  88,  90,  91,  92,  93,  95,  96,
                   97,  98, 100, 101, 102, 104, 105, 106, 107, 109, 110, 111, 112,
                  114, 115, 116, 117, 119, 120, 121, 122, 123, 125, 126, 127, 128,
                  130, 131, 132, 133, 135, 136, 137, 138, 140, 141, 142, 143, 145,
                  146, 147, 148, 150, 151, 152, 153, 155, 156, 157, 158, 160, 161,
                  162, 163, 165, 166, 167, 168, 170, 171, 172, 173, 175, 176, 177,
                  178, 180, 181, 182, 183, 185, 186, 187, 188, 190, 191, 192, 193,
                  195, 196, 197, 198]),
           array([  3,   8,  13,  18,  24,  29,  34,  39,  44,  49,  54,  59,  64,
                   69,  74,  79,  84,  89,  94,  99, 103, 108, 113, 118, 124, 129,
                  134, 139, 144, 149, 154, 159, 164, 169, 174, 179, 184, 189, 194,
                  199])),
          (array([  0,   1,   2,   3,   5,   6,   7,   8,  10,  11,  12,  13,  15,
                   16,  17,  18,  20,  21,  22,  24,  25,  26,  27,  28,  29,  31,
                   32,  33,  34,  37,  38,  39,  41,  42,  43,  44,  46,  47,  48,
                   49,  52,  53,  54,  56,  57,  58,  59,  61,  62,  63,  64,  66,
                   67,  68,  69,  71,  72,  73,  74,  75,  76,  77,  78,  79,  81,
                   82,  83,  84,  86,  87,  88,  89,  91,  92,  93,  94,  96,  97,
                   98,  99, 100, 101, 102, 103, 105, 106, 107, 108, 110, 111, 112,
                  113, 115, 116, 117, 118, 120, 121, 122, 124, 125, 126, 127, 128,
                  129, 131, 132, 133, 134, 137, 138, 139, 141, 142, 143, 144, 146,
                  147, 148, 149, 152, 153, 154, 156, 157, 158, 159, 161, 162, 163,
                  164, 166, 167, 168, 169, 171, 172, 173, 174, 175, 176, 177, 178,
                  179, 181, 182, 183, 184, 186, 187, 188, 189, 191, 192, 193, 194,
                  196, 197, 198, 199]),
           array([  4,   9,  14,  19,  23,  30,  35,  36,  40,  45,  50,  51,  55,
                   60,  65,  70,  80,  85,  90,  95, 104, 109, 114, 119, 123, 130,
                  135, 136, 140, 145, 150, 151, 155, 160, 165, 170, 180, 185, 190,
                  195])),
          (array([  0,   1,   2,   3,   4,   6,   7,   8,   9,  11,  12,  13,  14,
                   16,  17,  18,  19,  21,  23,  24,  25,  27,  28,  29,  30,  32,
                   33,  34,  35,  36,  38,  39,  40,  42,  43,  44,  45,  47,  48,
                   49,  50,  51,  52,  53,  54,  55,  57,  58,  59,  60,  62,  63,
                   64,  65,  67,  68,  69,  70,  72,  74,  76,  77,  78,  79,  80,
                   82,  83,  84,  85,  87,  88,  89,  90,  92,  93,  94,  95,  97,
                   98,  99, 100, 101, 102, 103, 104, 106, 107, 108, 109, 111, 112,
                  113, 114, 116, 117, 118, 119, 121, 123, 124, 125, 127, 128, 129,
                  130, 132, 133, 134, 135, 136, 138, 139, 140, 142, 143, 144, 145,
                  147, 148, 149, 150, 151, 152, 153, 154, 155, 157, 158, 159, 160,
                  162, 163, 164, 165, 167, 168, 169, 170, 172, 174, 176, 177, 178,
                  179, 180, 182, 183, 184, 185, 187, 188, 189, 190, 192, 193, 194,
                  195, 197, 198, 199]),
           array([  5,  10,  15,  20,  22,  26,  31,  37,  41,  46,  56,  61,  66,
                   71,  73,  75,  81,  86,  91,  96, 105, 110, 115, 120, 122, 126,
                  131, 137, 141, 146, 156, 161, 166, 171, 173, 175, 181, 186, 191,
                  196])),
          (array([  0,   2,   3,   4,   5,   7,   8,   9,  10,  12,  13,  14,  15,
                   17,  18,  19,  20,  22,  23,  24,  25,  26,  28,  29,  30,  31,
                   33,  34,  35,  36,  37,  38,  39,  40,  41,  43,  44,  45,  46,
                   48,  49,  50,  51,  53,  54,  55,  56,  58,  59,  60,  61,  63,
                   64,  65,  66,  68,  69,  70,  71,  73,  74,  75,  76,  78,  79,
                   80,  81,  83,  84,  85,  86,  88,  89,  90,  91,  93,  94,  95,
                   96,  99, 100, 102, 103, 104, 105, 107, 108, 109, 110, 112, 113,
                  114, 115, 117, 118, 119, 120, 122, 123, 124, 125, 126, 128, 129,
                  130, 131, 133, 134, 135, 136, 137, 138, 139, 140, 141, 143, 144,
```

```
        145, 146, 148, 149, 150, 151, 153, 154, 155, 156, 158, 159, 160,
        161, 163, 164, 165, 166, 168, 169, 170, 171, 173, 174, 175, 176,
        178, 179, 180, 181, 183, 184, 185, 186, 188, 189, 190, 191, 193,
        194, 195, 196, 199]),
 array([  1,   6,  11,  16,  21,  27,  32,  42,  47,  52,  57,  62,  67,
         72,  77,  82,  87,  92,  97,  98, 101, 106, 111, 116, 121, 127,
        132, 142, 147, 152, 157, 162, 167, 172, 177, 182, 187, 192, 197,
        198])),
(array([  1,   3,   4,   5,   6,   8,   9,  10,  11,  13,  14,  15,  16,
         18,  19,  20,  21,  22,  23,  24,  26,  27,  29,  30,  31,  32,
         34,  35,  36,  37,  39,  40,  41,  42,  44,  45,  46,  47,  49,
         50,  51,  52,  54,  55,  56,  57,  59,  60,  61,  62,  64,  65,
         66,  67,  69,  70,  71,  72,  73,  74,  75,  77,  79,  80,  81,
         82,  84,  85,  86,  87,  89,  90,  91,  92,  94,  95,  96,  97,
         98,  99, 101, 103, 104, 105, 106, 108, 109, 110, 111, 113, 114,
        115, 116, 118, 119, 120, 121, 122, 123, 124, 126, 127, 129, 130,
        131, 132, 134, 135, 136, 137, 139, 140, 141, 142, 144, 145, 146,
        147, 149, 150, 151, 152, 154, 155, 156, 157, 159, 160, 161, 162,
        164, 165, 166, 167, 169, 170, 171, 172, 173, 174, 175, 177, 179,
        180, 181, 182, 184, 185, 186, 187, 189, 190, 191, 192, 194, 195,
        196, 197, 198, 199]),
 array([  0,   2,   7,  12,  17,  25,  28,  33,  38,  43,  48,  53,  58,
         63,  68,  76,  78,  83,  88,  93, 100, 102, 107, 112, 117, 125,
        128, 133, 138, 143, 148, 153, 158, 163, 168, 176, 178, 183, 188,
        193]))]
```

In [ ]:

```python
# 1. Definimos los parámetros
params_baseline = {
    'num_classes': 2,
    'input_channels': input_channels,
    'seq_len': seq_len
}

# 2. Entrenamos usando la función run_kfold_experiment_pytorch
mean_acc, std_acc, histories, y_true, y_pred = run_kfold_experiment_pytorch(
    model_class=Baseline_CNN1D,
    model_params=params_baseline,
    X_full=X_tensor_concat,
    y_full=y_tensor_concat,
    groups=groups,
    epochs=100,
    batch_size=32,
    n_splits=5,
    patience=35,
    device='cpu'
)
```

```
🚀 Iniciando GroupKFold Modular (5 splits) en cpu...
   Protección contra Data Leakage activa (Grupos únicos detectados: 100)
============================================================

🔄 Pliegue 1/5
    ⏹️ Early stopping en época 82
    ✅ Fin Fold 1. Mejor Acc Val: 0.9250

🔄 Pliegue 2/5
    ⏹️ Early stopping en época 79
    ✅ Fin Fold 2. Mejor Acc Val: 0.9250

🔄 Pliegue 3/5
    ⏹️ Early stopping en época 52
    ✅ Fin Fold 3. Mejor Acc Val: 0.9250

🔄 Pliegue 4/5
    ✅ Fin Fold 4. Mejor Acc Val: 1.0000

🔄 Pliegue 5/5
    ✅ Fin Fold 5. Mejor Acc Val: 1.0000
============================================================
📊 Resultados GroupKFold:
| Accuracy Medio: 0.9550 (+/- 0.0367)
```

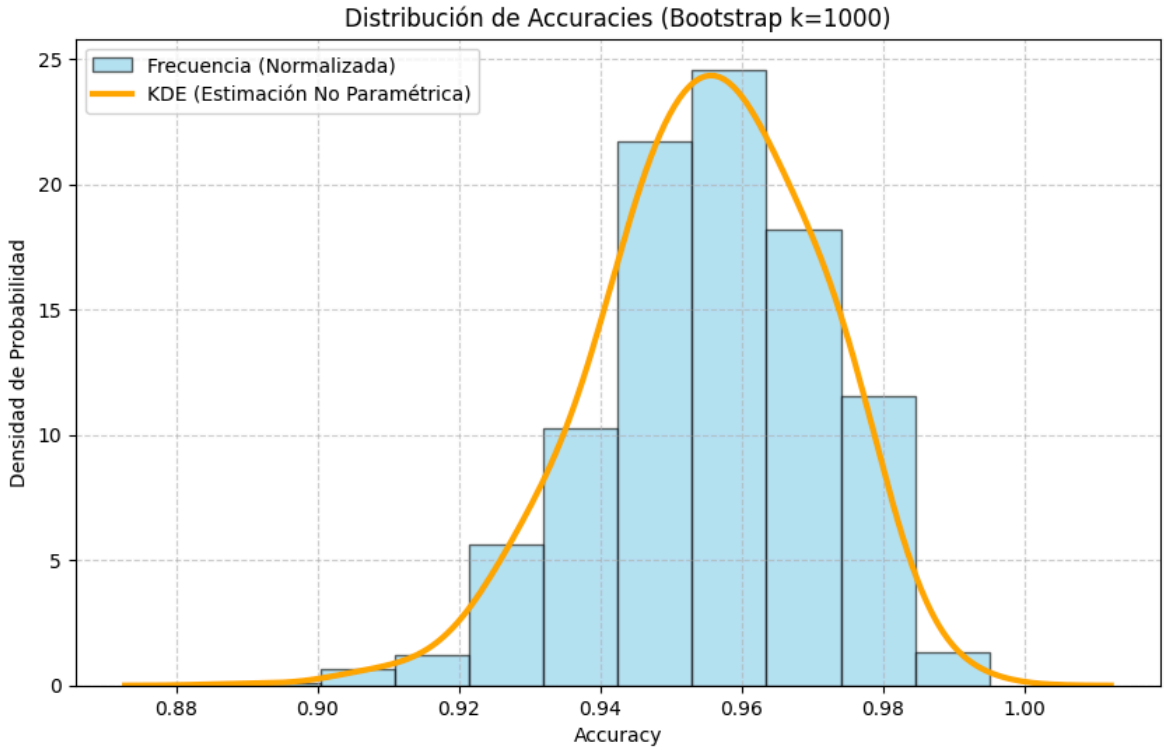
In [65]:
```
X_tensor_concat.shape
y_tensor_concat.shape
```

Out[65]:
```
torch.Size([200])
```

Evaluación con Bootstrapping

In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Graficamos el histograma de accuracies bootstrap
n_iterations = 1000
_, _, stats = calcular_bootstrap_intervalo(y_true, y_pred, n_iterations=n_iterat


plt.figure(figsize=(10, 6))

# Histograma NORMALIZADO (density=True)
plt.hist(stats, bins=10, color='skyblue', edgecolor='black',
         density=True, alpha=0.6, label='Frecuencia (Normalizada)')

# Gráfico KDE (Estimación de Densidad de Kernel)
sns.kdeplot(stats, color='orange', lw=3, label='KDE (Estimación No Paramétrica)'

plt.title(f'Distribución de Accuracies (Bootstrap k={n_iterations})')
plt.xlabel('Accuracy')
plt.ylabel('Densidad de Probabilidad') # Ojo: Ya no es "Frecuencia" absoluta
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

```
Accuracy Promedio (Bootstrap): 0.9551
Intervalo de Confianza 95%: [0.9250, 0.9800]
```



In [67]:
```
# Guardamos el array en disco para análisis posterior
np.save('../results/bootstrap_accuracies_exp_cnn_03.npy', stats)
```

EXP-CNN-04

In [ ]:
```
X_angles_correct_mirrored = np.load('../Data/Processed/mirrored_correct_push_ups
X_angles_incorrect_mirrored =ANGLES = np.load('../Data/Processed/mirrored_wrong_

X_angles_mirrored = np.vstack((X_angles_correct_mirrored, X_angles_incorrect_mir
y_angles_mirrored = np.array([1]*X_angles_correct_mirrored.shape[0] + [0]*X_angl

# Convertimos a tensores de PyTorch
X_angles_tensor_mirrored = torch.tensor(X_angles_mirrored, dtype=torch.float32)
y_angles_tensor_mirrored = torch.tensor(y_angles_mirrored, dtype=torch.long) # L

input_angles_channels = X_angles_tensor_mirrored.shape[2]  # Número de caracterí
seq_angles_len = X_angles_tensor_mirrored.shape[1]         # Número de frames po

# Obtenemos el número de muestras (N)
n_samples_angles_mirrored = X_angles_tensor_mirrored.shape[0]

# Creamos un array con los índices de los grupos (videos originales)
angles_mirrored_index = np.arange(n_samples_angles_mirrored)
```

In [69]:
```
# Creamos un array con los índices de los grupos (videos originales)
groups = np.concatenate([original_index, angles_mirrored_index])

# Concatenamos los datos originales y aumentados
X_tensor_concat = torch.cat((X_angles_tensor, X_angles_tensor_mirrored), dim=0)
y_tensor_concat = torch.cat((y_angles_tensor, y_angles_tensor_mirrored), dim=0)
```

In [ ]:
```
# 1. Definimos los parámetros
params_baseline = {
    'num_classes': 2,
```

```
    'input_channels': input_angles_channels,
    'seq_len': seq_angles_len
}

# 2. Entrenamos usando la función run_kfold_experiment_pytorch
mean_acc, std_acc, histories, y_true, y_pred = run_kfold_experiment_pytorch(
    model_class=Baseline_CNN1D,
    model_params=params_baseline,
    X_full=X_tensor_concat,
    y_full=y_tensor_concat,
    groups=groups,
    epochs=100,
    batch_size=32,
    n_splits=5,
    patience=35,
    device='cpu'
)
```

```
🚀 Iniciando GroupKFold Modular (5 splits) en cpu...
   Protección contra Data Leakage activa (Grupos únicos detectados: 100)
============================================================

🔄 Pliegue 1/5
    ✅ Fin Fold 1. Mejor Acc Val: 0.9000

🔄 Pliegue 2/5
    ✅ Fin Fold 2. Mejor Acc Val: 0.9750

🔄 Pliegue 3/5
    ⏹️ Early stopping en época 83
    ✅ Fin Fold 3. Mejor Acc Val: 0.8500

🔄 Pliegue 4/5
    ✅ Fin Fold 4. Mejor Acc Val: 0.9500

🔄 Pliegue 5/5
    ⏹️ Early stopping en época 50
    ✅ Fin Fold 5. Mejor Acc Val: 0.8750
============================================================
📊 Resultados GroupKFold:
| Accuracy Medio: 0.9100 (+/- 0.0464)
```

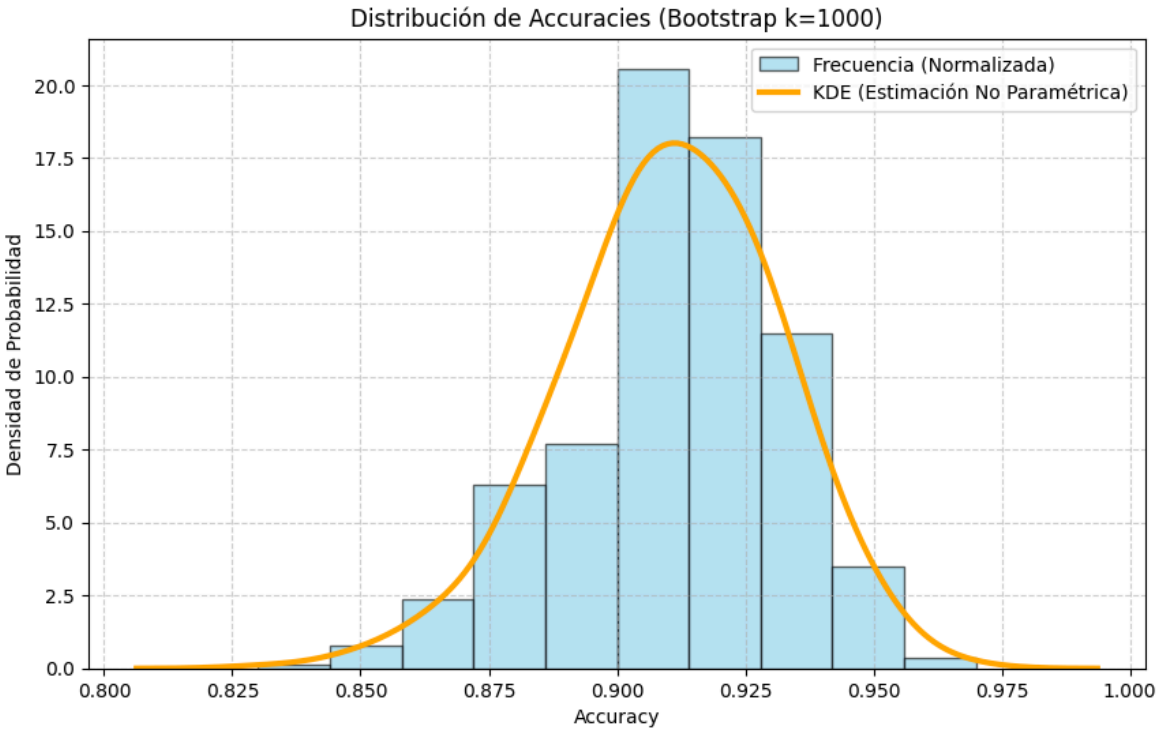
Evaluación con Bootstrapping

In [ ]:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Graficamos el histograma de accuracies bootstrap
n_iterations = 1000
_, _, stats = calcular_bootstrap_intervalo(y_true, y_pred, n_iterations=n_iterat


plt.figure(figsize=(10, 6))

# Histograma NORMALIZADO (density=True)
plt.hist(stats, bins=10, color='skyblue', edgecolor='black',
         density=True, alpha=0.6, label='Frecuencia (Normalizada)')

# Gráfico KDE (Estimación de Densidad de Kernel)
```

```
sns.kdeplot(stats, color='orange', lw=3, label='KDE (Estimación No Paramétrica)'

plt.title(f'Distribución de Accuracies (Bootstrap k={n_iterations})')
plt.xlabel('Accuracy')
plt.ylabel('Densidad de Probabilidad') # Ojo: Ya no es "Frecuencia" absoluta
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

```
Accuracy Promedio (Bootstrap): 0.9103
Intervalo de Confianza 95%: [0.8650, 0.9500]
```



In [72]:
```
# Guardamos el array en disco para análisis posterior
np.save('../results/bootstrap_accuracies_exp_cnn_04.npy', stats)
```

LSTM: Long Short-Term Memory

Librerias necesarias

In [1]:
```
# Librerias necesarias
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, KFold
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Masking, Bidirectional, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
```

Funciones

A continuación, definiremos las funciones que usaremos en el notebook

Esta primera función `plot_kfold_history_keras` grafica la pérdida y la precisión promedio de los 5 folds con sombra de desviación estándar.

In [ ]:
```
import matplotlib.pyplot as plt

def plot_kfold_history_keras(fold_histories, title='Métricas K-Fold LSTM'):
    """
    Grafica Loss y Accuracy promedio de los 5 folds con sombra de desviación est
    """

    if not fold_histories:
        print("Error: No hay historias para graficar.")
        return

    # Encontramos la longitud mínima (por si hubo Early Stopping en tiempos dist
    # Cortamos todas las gráficas a la longitud del entrenamiento más corto
    min_epochs = min(len(h['loss']) for h in fold_histories)

    print(f"Graficando las primeras {min_epochs} épocas comunes")

    # Creamos arrays para promedios
    metrics_map = {
        'loss': 'loss',
        'val_loss': 'val_loss',
        'acc': 'accuracy',
        'val_acc': 'val_accuracy'
    }

    data = {k: np.zeros((len(fold_histories), min_epochs)) for k in metrics_map.

    # Rellenamos datos recortando al mínimo común
    for i, history in enumerate(fold_histories):
        for key_out, key_in in metrics_map.items(): # key_out es 'acc' (para el
            if key_in in history:
                data[key_out][i, :] = history[key_in][:min_epochs]
            else:
                print(f"Métrica '{key_in}' no encontrada en el historial.")

    # 4. Calculamos medias y desviaciones
    means = {m: np.mean(data[m], axis=0) for m in data}
    stds = {m: np.std(data[m], axis=0) for m in data}
    epochs_range = range(1, min_epochs + 1)

    # PLOT 1: LOSS
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    # Train Loss
    plt.plot(epochs_range, means['loss'], label='Train Loss', color='blue')
    plt.fill_between(epochs_range, means['loss'] - stds['loss'], means['loss'] +
    # Val Loss
    plt.plot(epochs_range, means['val_loss'], label='Val Loss', color='red')
    plt.fill_between(epochs_range, means['val_loss'] - stds['val_loss'], means['

    plt.title(f'{title} - Pérdida')
    plt.xlabel('Época')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.6)
```

```
    # PLOT 2: ACCURACY
    plt.subplot(1, 2, 2)
    # Train Acc
    plt.plot(epochs_range, means['acc'], label='Train Acc', color='blue')
    plt.fill_between(epochs_range, means['acc'] - stds['acc'], means['acc'] + st
    # Val Acc
    plt.plot(epochs_range, means['val_acc'], label='Val Acc', color='green')
    plt.fill_between(epochs_range, means['val_acc'] - stds['val_acc'], means['va

    plt.title(f'{title} - Accuracy')
    plt.xlabel('Época')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.6)

    plt.tight_layout()
    plt.show()
```

La función `graficar_matriz_confusion` recibe las predicciones y etiquetas reales y genera la matriz

In [ ]:
```
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

def graficar_matriz_confusion(y_true, y_pred, nombre_modelo="Modelo"):
    """
    Recibe las predicciones y etiquetas reales y pinta la matriz.
    """
    print(f"Generando Matriz de Confusión para: {nombre_modelo}")

    # Aseguramos enteros
    y_true_int = y_true.astype(int)
    y_pred_int = (y_pred > 0.5).astype(int)

    # Calculamos la matriz de confusión
    cm = confusion_matrix(y_true_int, y_pred_int)

    # Graficamos la matriz de confusión
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
                xticklabels=['Incorrecta', 'Correcta'],
                yticklabels=['Incorrecta', 'Correcta'])
    plt.title(f'Matriz de Confusión: {nombre_modelo}')
    plt.ylabel('Realidad')
    plt.xlabel('Predicción')
    plt.show()
```

Definimos las siguientes funciones:

- `train_single_fold` : se encarga de crear, entrenar, evaluar y predecir para UN solo pliegue.
- `run_kfold_experiment` : se encarga de rrquestar el GroupKFold para evitar Data Leakage con datos aumentados.

In [ ]:
```python
import numpy as np
from sklearn.model_selection import KFold, GroupKFold
from tensorflow.keras.callbacks import EarlyStopping

# 1. GESTOR DE UN SOLO PLIEGUE (FOLD MANAGER)
def train_single_fold(model_fn, X_train_f, y_train_f, X_val_f, y_val_f,
                      timesteps, features, epochs, batch_size, patience, fold_n)
    """
    Se encarga de crear, entrenar, evaluar y predecir para UN solo pliegue.
    """

    # Configuramos Callbacks
    callbacks_list = []
    if patience is not None and patience > 0:
        early_stopping = EarlyStopping(
            monitor='val_loss',
            patience=patience,
            restore_best_weights=True,
            verbose=0
        )
        callbacks_list.append(early_stopping)

    # Instanciamos Modelo (Limpio)
    model = model_fn(timesteps, features)

    # Entrenamos
    history = model.fit(
        X_train_f, y_train_f,
        validation_data=(X_val_f, y_val_f),
        epochs=epochs,
        batch_size=batch_size,
        callbacks=callbacks_list,
        verbose=0 # Silencioso para no saturar la consola
    )

    # Evaluamos Métricas
    loss, acc = model.evaluate(X_val_f, y_val_f, verbose=0)

    # Generamos Predicciones (Inferencia)
    y_pred_prob = model.predict(X_val_f, verbose=0)

    # Conversión Probabilidad -> Clase
    if y_pred_prob.shape[-1] == 1:
        # Caso Binario Sigmoid
        y_pred_class = (y_pred_prob > 0.5).astype(int).flatten()
    else:
        # Caso Softmax
        y_pred_class = np.argmax(y_pred_prob, axis=-1)

    return acc, loss, history.history, y_pred_class


# 2. ORQUESTADOR PRINCIPAL (GROUP K-FOLD)
def run_kfold_experiment(model_fn, X_train, y_train, groups,
                                    epochs, batch_size, n_splits=5, patience=Non
    """
    Orquesta el GroupKFold para evitar Data Leakage con datos aumentados.

    Args:
```

```
    groups (np.array): Array de IDs que vincula muestras originales con sus
"""

# Usamos GroupKFold en lugar de KFold estándar
gkf = GroupKFold(n_splits=n_splits)

# Acumuladores
fold_accuracies = []
fold_histories = []

# Listas para reordenamiento
global_y_true_unsorted = []
global_y_pred_unsorted = []
global_indices = []

print(f"GroupKFold Keras ({n_splits} splits)")
print(f"(Grupos detectados: {len(np.unique(groups))})")
print("===" * 20)

_, timesteps, features = X_train.shape

# Pasamos 'groups' al split
for fold_idx, (train_index, val_index) in enumerate(gkf.split(X_train, y_tra
    print(f"\nPliegue {fold_idx + 1}/{n_splits}")

    # Preparamos los datos
    X_train_f, X_val_f = X_train[train_index], X_train[val_index]
    y_train_f, y_val_f = y_train[train_index], y_train[val_index]

    # Delegamos trabajo a train_single_fold
    acc, loss, history, fold_preds = train_single_fold(
        model_fn, X_train_f, y_train_f, X_val_f, y_val_f,
        timesteps, features, epochs, batch_size, patience, fold_idx+1
    )

    print(f"Fin Fold {fold_idx+1}. Acc Val: {acc:.4f} | Loss: {loss:.4f}")

    # Recolectamos resultados
    fold_accuracies.append(acc)
    fold_histories.append(history)

    global_y_true_unsorted.extend(y_val_f)
    global_y_pred_unsorted.extend(fold_preds)
    global_indices.extend(val_index)

# Reordenamiento final
global_indices_np = np.array(global_indices)
sorted_order = np.argsort(global_indices_np)

final_y_true = np.array(global_y_true_unsorted)[sorted_order]
final_y_pred = np.array(global_y_pred_unsorted)[sorted_order]

mean_acc = np.mean(fold_accuracies)
std_acc = np.std(fold_accuracies)

print("===" * 20)
print(f"Resultados GroupKFold:")
print(f"Accuracy Medio: {mean_acc:.4f}")
print(f"Desviación Std: {std_acc:.4f}")
```

```
    return mean_acc, std_acc, fold_histories, final_y_true, final_y_pred
```

La función `calcular_bootstrap_intervalo` calcula el intervalo de confianza usando Bootstrapping sobre las predicciones.

In [ ]:
```
from sklearn.utils import resample
from sklearn.metrics import confusion_matrix, classification_report, accuracy_sc

def calcular_bootstrap_intervalo(y_true, y_pred, n_iterations=1000, alpha=0.95):
    """
    Calcula el intervalo de confianza usando Bootstrapping sobre las prediccione
    """
    stats = []

    # Bucle de simulaciones (1000 veces)
    for i in range(n_iterations):
        # Generamos índices aleatorios con reemplazo
        # (Simula meter la mano en la bolsa y sacar bolas repetidas)
        indices = resample(range(len(y_pred)), replace=True)

        # Creamos el "dataset virtual"
        sample_pred = y_pred[indices]
        sample_true = y_true[indices]

        # Calculamos el accuracy de esta simulación
        score = accuracy_score(sample_true, sample_pred)
        stats.append(score)

    # 2. Ordenamos los 1000 resultados de menor a mayor
    stats = np.sort(stats)

    # 3. Calculamos los límites (cortamos las colas)
    lower_percentile = ((1.0 - alpha) / 2.0) * 100
    upper_percentile = (alpha + ((1.0 - alpha) / 2.0)) * 100

    lower = np.percentile(stats, lower_percentile)
    upper = np.percentile(stats, upper_percentile)

    print(f"Accuracy Promedio (Bootstrap): {np.mean(stats):.4f}")
    print(f"Intervalo de Confianza {int(alpha*100)}%: [{lower:.4f}, {upper:.4f}]

    return lower, upper, stats
```

Con `create_custom_lstm` creamos lstm con personalizadas, indicando los parámetros que desamos que tengan.

In [ ]:
```
from tensorflow.keras.optimizers import Adam, RMSprop, SGD

def create_custom_lstm(timesteps, features,
                       lstm_layers=[64], # lstm_layers: [128, 64] crea dos capas
                       dense_layers=[32], # dense_layers: [64, 32] crea dos capa
                       dropout_rate=0.2,
                       learning_rate=0.001,
                       bidirectional=False,
                       optimizer_name='adam'): # Puede ser 'adam', 'sgd', 'rmspr

    model = Sequential()
```

```
    model.add(Masking(mask_value=0., input_shape=(timesteps, features)))

    # Bucle para CAPAS LSTM (Automático: detecta si debe devolver secuencias). C
    for i, units in enumerate(lstm_layers):
            is_last_lstm = (i == len(lstm_layers) - 1)

            # Definimos la capa base
            lstm_layer = LSTM(units, return_sequences=not is_last_lstm)

            # Decidimos si la envolvemos en Bidirectional o no
            if bidirectional:
                model.add(Bidirectional(lstm_layer))
            else:
                model.add(lstm_layer)

            if dropout_rate > 0:
                model.add(Dropout(dropout_rate))

    # Bucle para CAPAS DENSAS.
    for units in dense_layers:
        model.add(Dense(units, activation='relu'))
        if dropout_rate > 0:
            model.add(Dropout(dropout_rate))

    # Capa de Salida
    model.add(Dense(1, activation='sigmoid'))

    # Selector de Optimizador
    if optimizer_name.lower() == 'adam':
        opt = Adam(learning_rate=learning_rate)
    elif optimizer_name.lower() == 'rmsprop':
        opt = RMSprop(learning_rate=learning_rate)
    elif optimizer_name.lower() == 'sgd':
        opt = SGD(learning_rate=learning_rate)
    else:
        opt = Adam(learning_rate=learning_rate)

    model.compile(loss='binary_crossentropy', optimizer=opt, metrics=['accuracy'
    return model
```

Por último, `ejecutar_experimento_individual` entrena con GroupKFold, calcula Bootstrap, grafica el resultado y guarda los datos.

In [ ]:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Inicializamos la lista que contendrá los resultados (Globales)
results_table = []

# Definimos la función que hace todo (Entrenar + Bootstrap + Graficar)
def ejecutar_experimento_individual(name, lstm_layers, dense_layers, dropout, lr
    """
    1. Entrena con GroupKFold.
    2. Calcula Bootstrap.
    3. GRAFICA el resultado inmediatamente.
    4. Guarda los datos en la tabla global.
    """
```

```
    print(f"Iniciando: {name}")
    print(f"Arq: LSTM={lstm_layers} (Bi={bidirectional}) | Dense={dense_layers}"
    print(f"Params: Opt={opt_name} | LR={lr} | Drop={dropout} | Patience={patien
    print("-" * 60)

    # Construimos el modelo con los parámetros pasados
    model_builder = lambda t, f: create_custom_lstm(
        t, f,
        lstm_layers=lstm_layers,
        dense_layers=dense_layers,
        dropout_rate=dropout,
        learning_rate=lr,
        optimizer_name=opt_name
    )

    # Ejecutamos el K-Fold pasando el 'patience' para el Early Stopping
    mean_acc, std_acc, histories, final_y_true, final_y_pred = run_kfold_experim
        model_fn=model_builder,
        X_train=X_total, y_train=y_total, groups=groups,
        epochs=epochs, batch_size=batch_size, n_splits=5,
        patience=patience
    )

    # Calculamos Bootstrap
    print(f"Calculando Bootstrap (10,000 iteraciones)")
    n_iterations = 10000
    lower, upper, stats = calcular_bootstrap_intervalo(
        final_y_true, final_y_pred, n_iterations=n_iterations, alpha=0.95
    )

    boot_mean = np.mean(stats) # Calculamos la media de las simulaciones

    print(f"Resultado: Acc Promedio {boot_mean:.4f} | IC 95%: [{lower:.4f}, {upp

    # Gráficas de entrenamiento
    print("\nCurvas de aprendizaje promedio:")
    plot_kfold_history_keras(histories, title=f'Modelo: {name}')

    # Gráficas de Bootstrap
    plt.figure(figsize=(10, 6))

    # 1. Histograma normalizado
    plt.hist(stats, bins=30, color='skyblue', edgecolor='black',
             density=True, alpha=0.6, label='Frecuencia (Normalizada)')

    # 2. Gráfico KDE
    sns.kdeplot(stats, color='orange', lw=3, label='KDE (Estimación No Paramétri

    # Líneas verticales para el intervalo de confianza (Opcional, pero recomenda
    plt.axvline(lower, color='red', linestyle='--', alpha=0.8, label=f'Límite In
    plt.axvline(upper, color='green', linestyle='--', alpha=0.8, label=f'Límite

    plt.title(f'Distribución de Accuracies - Modelo: {name}\n(Bootstrap k={n_ite
    plt.xlabel('Accuracy')
    plt.ylabel('Densidad de Probabilidad')
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.show()

    # Guardamos en tabla global
```

```
    # Borramos si ya existía este modelo para actualizarlo
    global results_table
    results_table = [r for r in results_table if r['Modelo'] != name]
    results_table.append({
        'Modelo': name,
        'Acc CV': mean_acc,
        'Acc Boot': boot_mean,
        'IC 95%': f"[{lower:.3f}-{upper:.3f}]",
        'LSTM Layers': str(lstm_layers), # Lo guardamos como texto para leerlo m
        'Bidirectional': bidirectional,
        'Dense Layers': str(dense_layers),
        'Optimizador': opt_name,
        'Batch_size' : batch_size,
        'Epochs' : epochs,
        'LR': lr,
        'Dropout': dropout,
        'Patience': patience
    })

    # 3. Guardamos los resultados de Bootstrap en un archivo .npy para análisis
    filename = f"../results/bootstrap_accuracies_exp_{name}.npy"
    np.save(filename, stats)

    return final_y_true, final_y_pred # Devuelve los valores reales y predichos
```

Experimentación con aumentacion con Landmarks

Usaremos los ficheros de datos con puntos clave (originales y espejados)

In [ ]:
```
import numpy as np

# Cargamos los datos
X_correct_mirrored = np.load('../Data/Processed/mirrored_correct_push_ups_landma
X_incorrect_mirrored = np.load('../Data/Processed/mirrored_wrong_push_ups_landma
X_correct_original = np.load('../Data/Processed/correct_push_ups_landmarks.npy')
X_incorrect_original = np.load('../Data/Processed/wrong_push_ups_landmarks.npy')

# Concatenamos clases para formar X e y (Mirrored y Original por separado primer
# MIRRORED
X_mirrored = np.vstack((X_correct_mirrored, X_incorrect_mirrored))
y_mirrored = np.array([1]*len(X_correct_mirrored) + [0]*len(X_incorrect_mirrored

# ORIGINAL
X_original = np.vstack((X_correct_original, X_incorrect_original))
y_original = np.array([1]*len(X_correct_original) + [0]*len(X_incorrect_original

# Creamos los grupos
# El video i-ésimo de X_original corresponde al video i-ésimo de X_mirrored
# Creamos un ID único para cada video original
original_ids = np.arange(len(X_original))
mirrored_ids = np.arange(len(X_mirrored)) # Igual que el anterior

# Juntamos todo (Original + Mirrored)
X_total = np.concatenate((X_original, X_mirrored), axis=0)
y_total = np.concatenate((y_original, y_mirrored), axis=0)
```

```
# El array 'groups' le dice al KFold qué filas son "hermanas"
groups = np.concatenate((original_ids, mirrored_ids), axis=0) # Evita fugas de a

print(f"Forma de X_total: {X_total.shape}")
print(f"Forma de y_total: {y_total.shape}")
print(f"Forma de groups: {groups.shape}")
print(f"Ejemplo: El dato en índice 0 (orig) y el índice {len(X_original)} (mirro
```

```
Forma de X_total: (200, 160, 66)
Forma de y_total: (200,)
Forma de groups: (200,)
Ejemplo: El dato en índice 0 (orig) y el índice 100 (mirror) tienen el grupo 0 y
0
```

In [9]:
```
groups
```

Out[9]:
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
       34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50,
       51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67,
       68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84,
       85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99,  0,  1,
        2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17, 18,
       19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35,
       36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52,
       53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69,
       70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86,
       87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

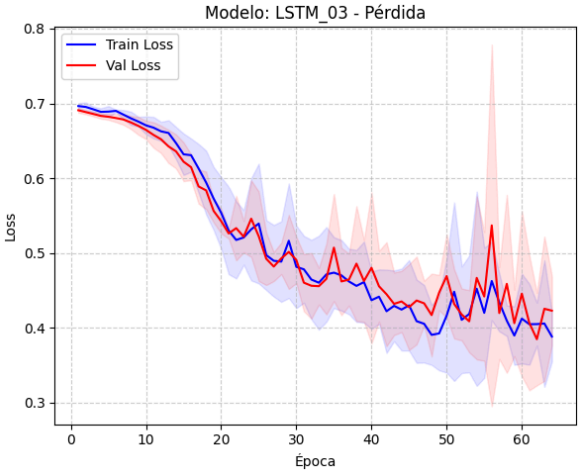
In [10]:
```
# Definimos parámetros
TIMESTEPS = X_total.shape[1]
FEATURES = X_total.shape[2]
```

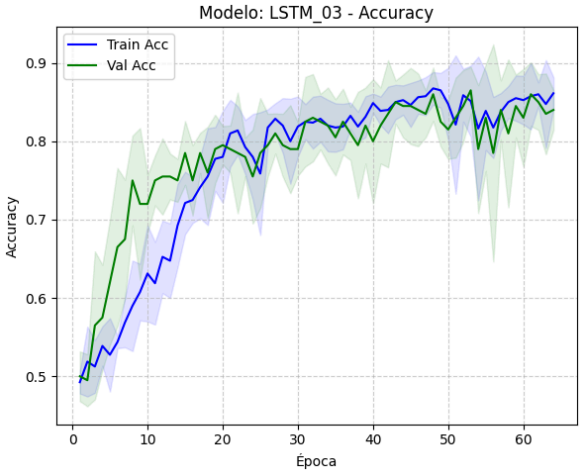
Exp-LSTM-03

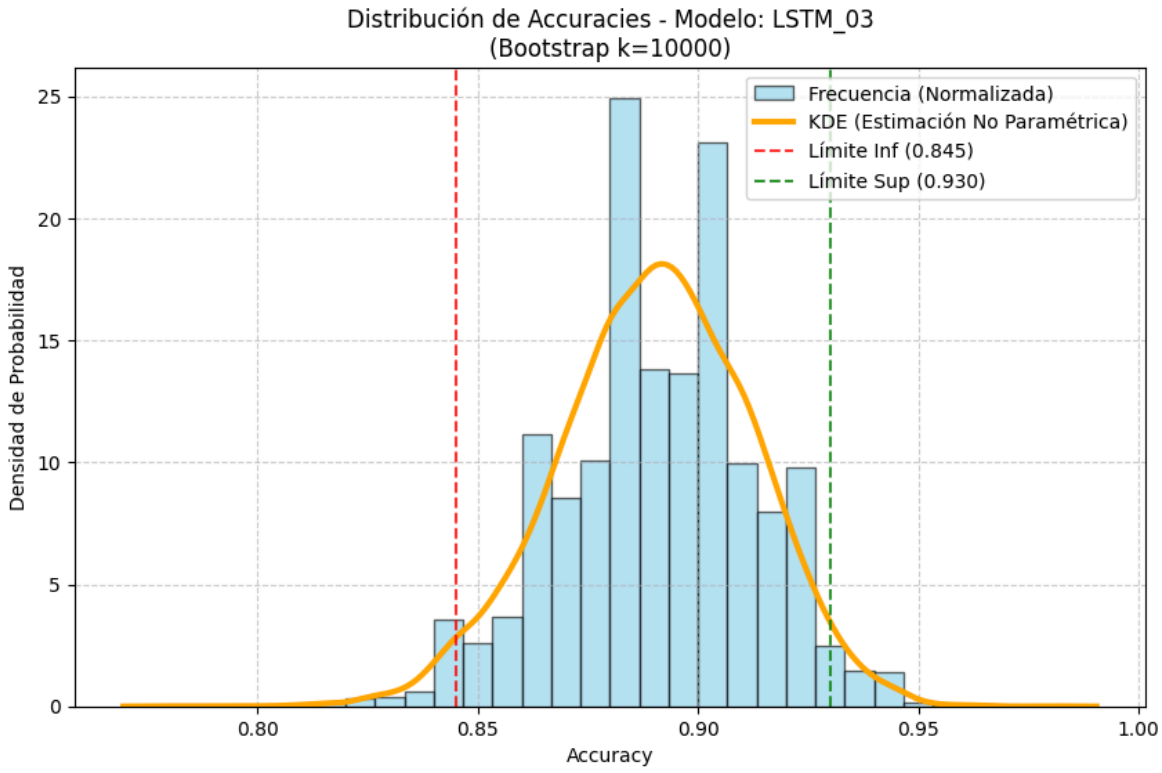
In [ ]:
```
y_real, y_predicha = ejecutar_experimento_individual( # LSTM_LongTrain
    name='LSTM_03',
    lstm_layers=[128, 64],
    dense_layers=[64, 32],
    dropout=0.2,
    lr=0.00005,
    opt_name='adam',
    patience=15,
    epochs=100
)
```

```
Iniciando: LSTM_03
Arq: LSTM=[128, 64] (Bi=False) | Dense=[64, 32]
Params: Opt=adam | LR=5e-05 | Drop=0.2 | Patience=15
------------------------------------------------------------
GroupKFold Keras (5 splits)
(Grupos detectados: 100)
============================================================

Pliegue 1/5
```

```
/opt/miniconda3/envs/exml-py310/lib/python3.10/site-packages/keras/src/layers/core/masking.py:48: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
Fin Fold 1. Acc Val: 0.8500 | Loss: 0.4269

Pliegue 2/5
Fin Fold 2. Acc Val: 0.9000 | Loss: 0.2715

Pliegue 3/5
Fin Fold 3. Acc Val: 0.9250 | Loss: 0.2415

Pliegue 4/5
Fin Fold 4. Acc Val: 0.8500 | Loss: 0.4065

Pliegue 5/5
Fin Fold 5. Acc Val: 0.9250 | Loss: 0.2132
============================================================
Resultados GroupKFold:
Accuracy Medio: 0.8900
Desviación Std: 0.0339
Calculando Bootstrap (10,000 iteraciones)
Accuracy Promedio (Bootstrap): 0.8901
Intervalo de Confianza 95%: [0.8450, 0.9300]
Resultado: Acc Promedio 0.8901 | IC 95%: [0.8450, 0.9300]

Curvas de aprendizaje promedio:
Graficando las primeras 64 épocas comunes
```

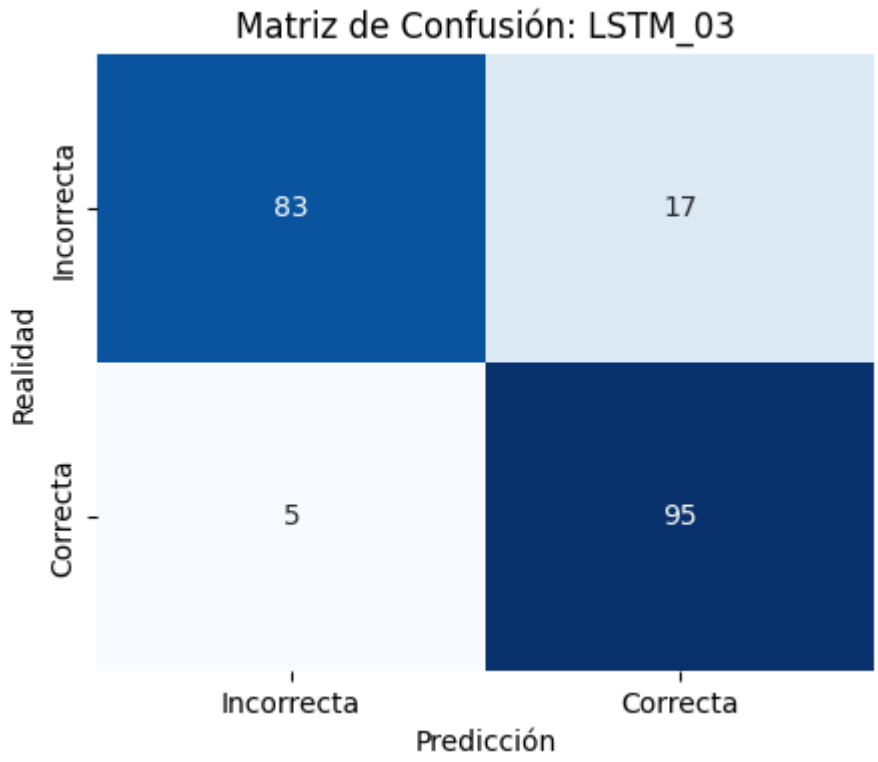






In [13]:
```
graficar_matriz_confusion(y_real, y_predicha, nombre_modelo="LSTM_03")
```

```
Generando Matriz de Confusión para: LSTM_03
```

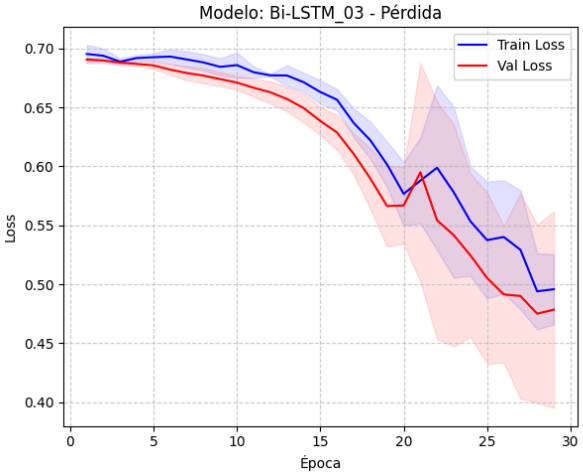


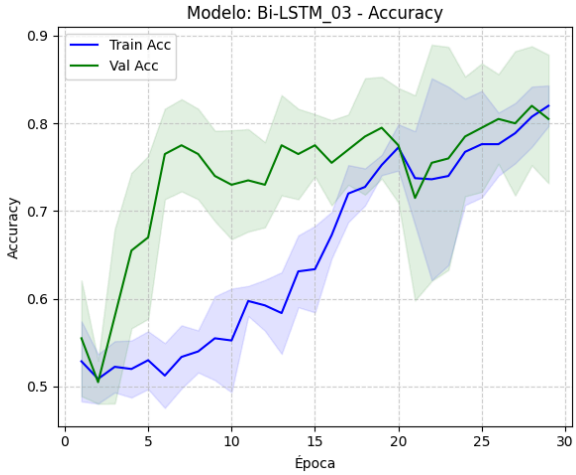
Exp-Bi-LSTM-03

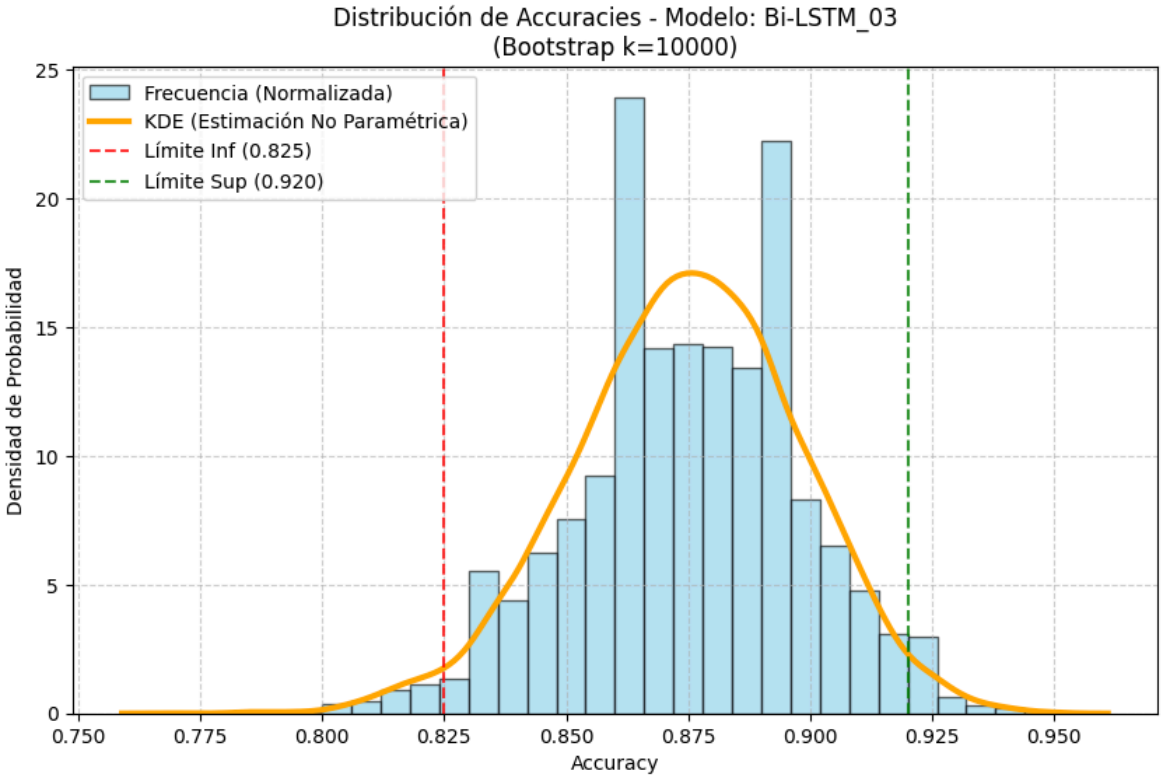
In [ ]:
```
_, _ = ejecutar_experimento_individual(
    name='Bi-LSTM_03',
    lstm_layers=[128, 64],
    dense_layers=[64, 32],
    dropout=0.25,
    lr=0.00005,
```

```
    opt_name='adam',
    bidirectional=True, # LSTM bidireccional
    patience=10,
    epochs=80
)
```

```
Iniciando: Bi-LSTM_03
Arq: LSTM=[128, 64] (Bi=True) | Dense=[64, 32]
Params: Opt=adam | LR=5e-05 | Drop=0.25 | Patience=10
------------------------------------------------------------
GroupKFold Keras (5 splits)
(Grupos detectados: 100)
============================================================

Pliegue 1/5
/opt/miniconda3/envs/exml-py310/lib/python3.10/site-packages/keras/src/layers/core/masking.py:48: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
Fin Fold 1. Acc Val: 0.8500 | Loss: 0.4189

Pliegue 2/5
Fin Fold 2. Acc Val: 0.9000 | Loss: 0.2592

Pliegue 3/5
Fin Fold 3. Acc Val: 0.8750 | Loss: 0.5148

Pliegue 4/5
Fin Fold 4. Acc Val: 0.8000 | Loss: 0.4843

Pliegue 5/5
Fin Fold 5. Acc Val: 0.9500 | Loss: 0.2264
============================================================
Resultados GroupKFold:
Accuracy Medio: 0.8750
Desviación Std: 0.0500
Calculando Bootstrap (10,000 iteraciones)
Accuracy Promedio (Bootstrap): 0.8746
Intervalo de Confianza 95%: [0.8250, 0.9200]
Resultado: Acc Promedio 0.8746 | IC 95%: [0.8250, 0.9200]

Curvas de aprendizaje promedio:
Graficando las primeras 29 épocas comunes
```







Experimentación con aumentación y datos angulares

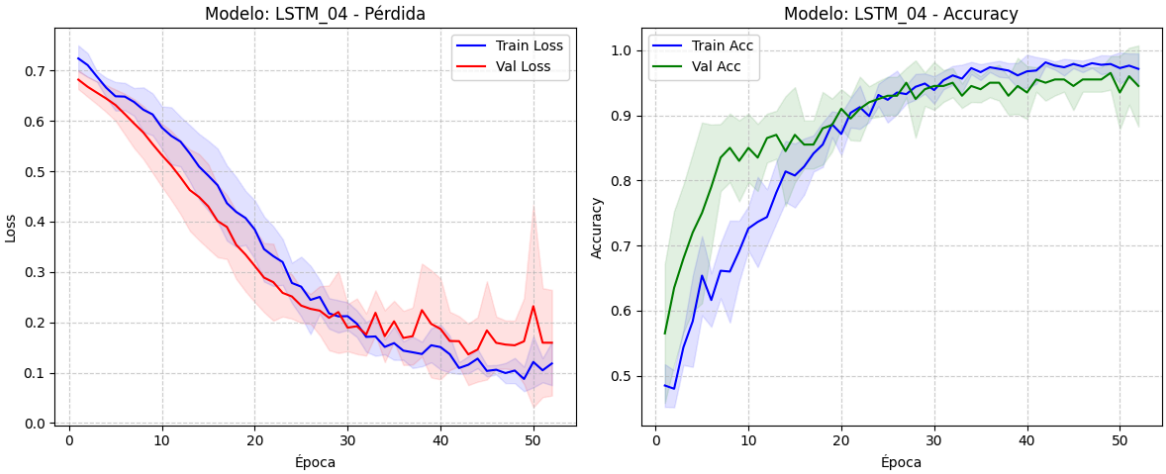
In [ ]:
```
import numpy as np

# Cargamos los datos
X_correct_mirrored = np.load('../Data/Processed/mirrored_correct_push_ups_angles
X_incorrect_mirrored = np.load('../Data/Processed/mirrored_wrong_push_ups_angles
X_correct_original = np.load('../Data/Processed/correct_push_ups_angles.npy')
X_incorrect_original = np.load('../Data/Processed/wrong_push_ups_angles.npy')

# MIRRORED
X_mirrored = np.vstack((X_correct_mirrored, X_incorrect_mirrored))
y_mirrored = np.array([1]*len(X_correct_mirrored) + [0]*len(X_incorrect_mirrored

# ORIGINAL
X_original = np.vstack((X_correct_original, X_incorrect_original))
y_original = np.array([1]*len(X_correct_original) + [0]*len(X_incorrect_original

# Creación de los grupos
original_ids = np.arange(len(X_original))
mirrored_ids = np.arange(len(X_mirrored)) # Debería ser igual al anterior si est

# Juntamos todo (Original + Mirrored)
X_total = np.concatenate((X_original, X_mirrored), axis=0)
y_total = np.concatenate((y_original, y_mirrored), axis=0)

groups = np.concatenate((original_ids, mirrored_ids), axis=0)

print(f"Forma de X_total: {X_total.shape}")
print(f"Forma de y_total: {y_total.shape}")
print(f"Forma de groups: {groups.shape}")
print(f"Ejemplo: El dato en índice 0 (orig) y el índice {len(X_original)} (mirro
```

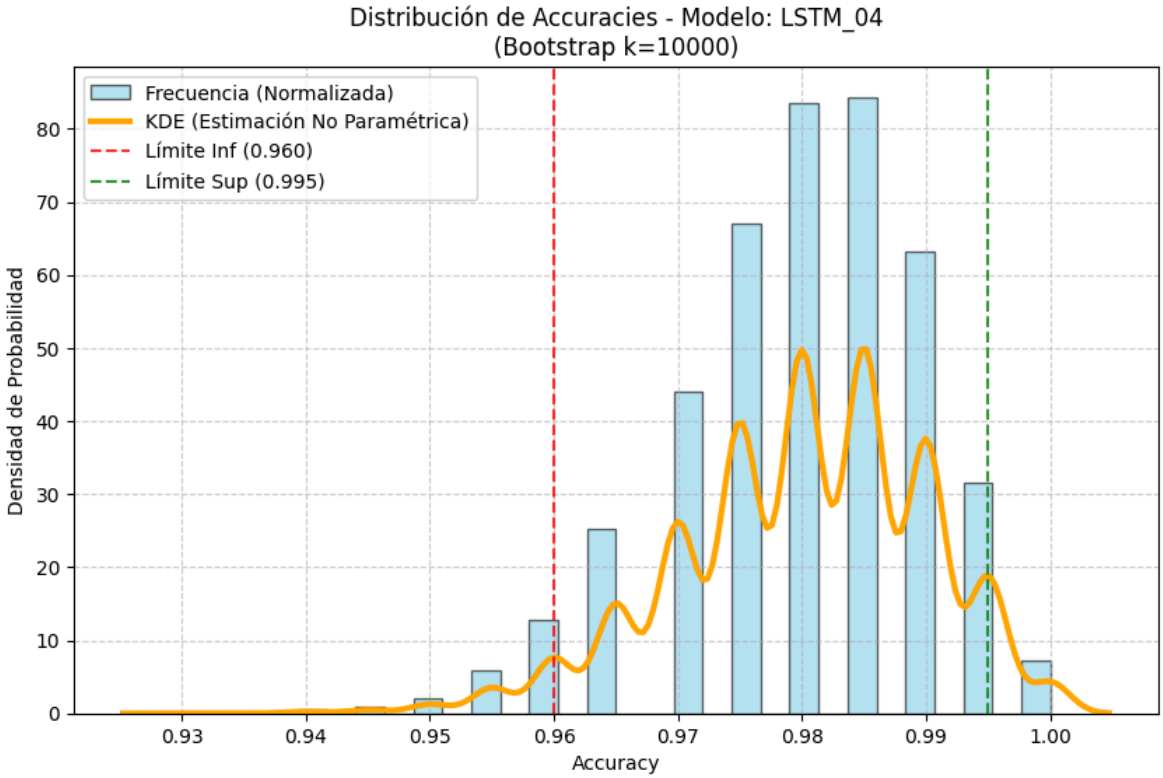
```
Forma de X_total: (200, 160, 10)
Forma de y_total: (200,)
Forma de groups: (200,)
Ejemplo: El dato en índice 0 (orig) y el índice 100 (mirror) tienen el grupo 0 y
0
```

Exp-LSTM-04

In [ ]:
```python
y_real, y_predicha = ejecutar_experimento_individual( #LSTM_Wide_Angles
    name='LSTM_04',
    lstm_layers=[256, 128],
    dense_layers=[64, 32],
    dropout=0.3,
    lr=0.00005,
    opt_name='adam',
    patience=15,
    epochs=80
)
```

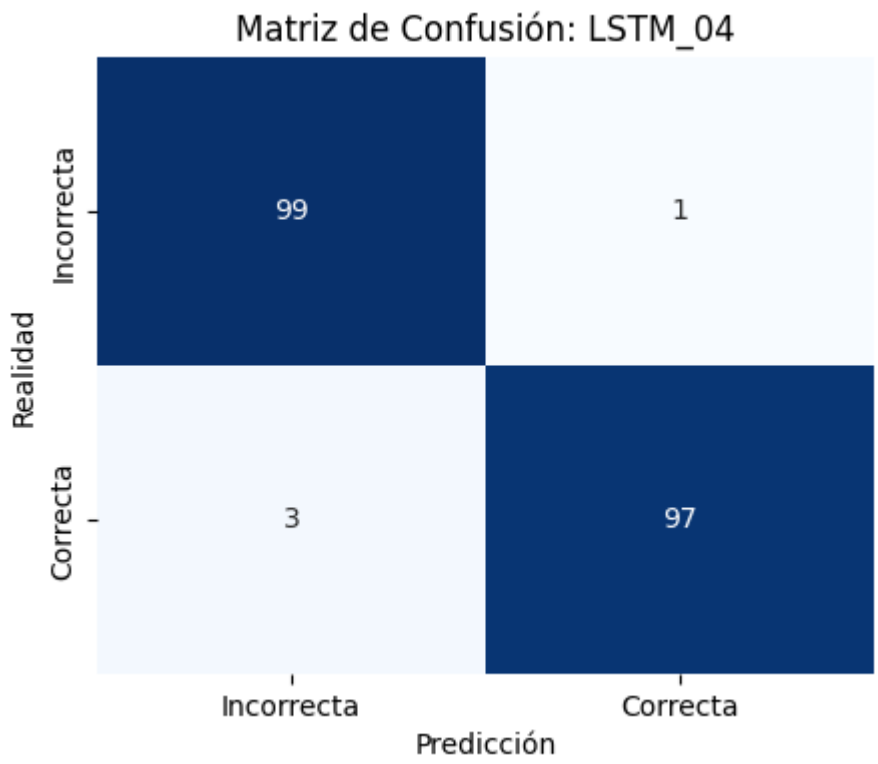
```
Iniciando: LSTM_04
Arq: LSTM=[256, 128] (Bi=False) | Dense=[64, 32]
Params: Opt=adam | LR=5e-05 | Drop=0.3 | Patience=15
------------------------------------------------------------
GroupKFold Keras (5 splits)
(Grupos detectados: 100)
============================================================

Pliegue 1/5
/opt/miniconda3/envs/exml-py310/lib/python3.10/site-packages/keras/src/layers/cor
e/masking.py:48: UserWarning: Do not pass an `input_shape`/`input_dim` argument t
o a layer. When using Sequential models, prefer using an `Input(shape)` object as
the first layer in the model instead.
  super().__init__(**kwargs)
Fin Fold 1. Acc Val: 0.9750 | Loss: 0.0799

Pliegue 2/5
Fin Fold 2. Acc Val: 0.9750 | Loss: 0.1313

Pliegue 3/5
Fin Fold 3. Acc Val: 1.0000 | Loss: 0.0172

Pliegue 4/5
Fin Fold 4. Acc Val: 0.9500 | Loss: 0.2175

Pliegue 5/5
Fin Fold 5. Acc Val: 1.0000 | Loss: 0.0340
============================================================
Resultados GroupKFold:
Accuracy Medio: 0.9800
Desviación Std: 0.0187
Calculando Bootstrap (10,000 iteraciones)
Accuracy Promedio (Bootstrap): 0.9800
Intervalo de Confianza 95%: [0.9600, 0.9950]
Resultado: Acc Promedio 0.9800 | IC 95%: [0.9600, 0.9950]

Curvas de aprendizaje promedio:
Graficando las primeras 52 épocas comunes
```





In [17]:
```
graficar_matriz_confusion(y_real, y_predicha, nombre_modelo="LSTM_04")
```

```
Generando Matriz de Confusión para: LSTM_04
```

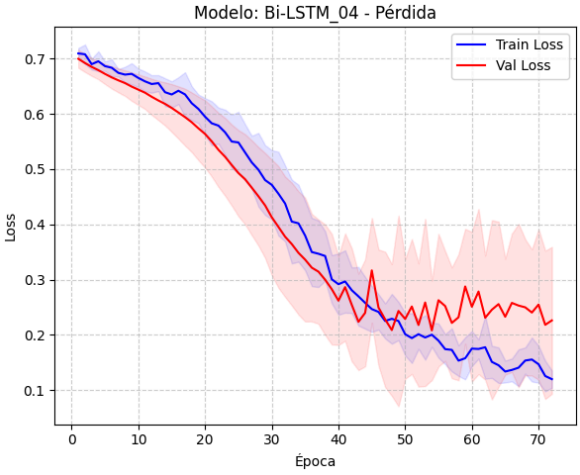


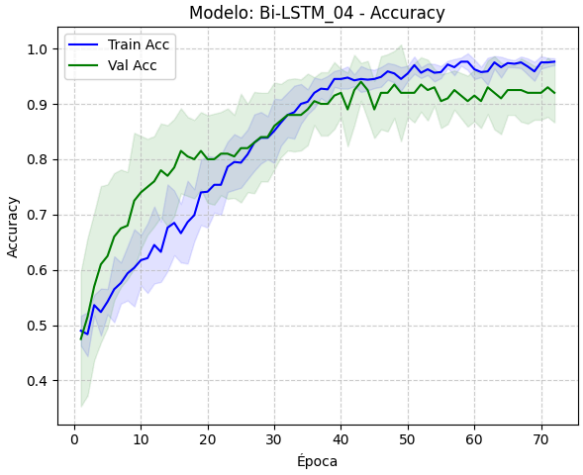
Exp-Bi-LSTM-04

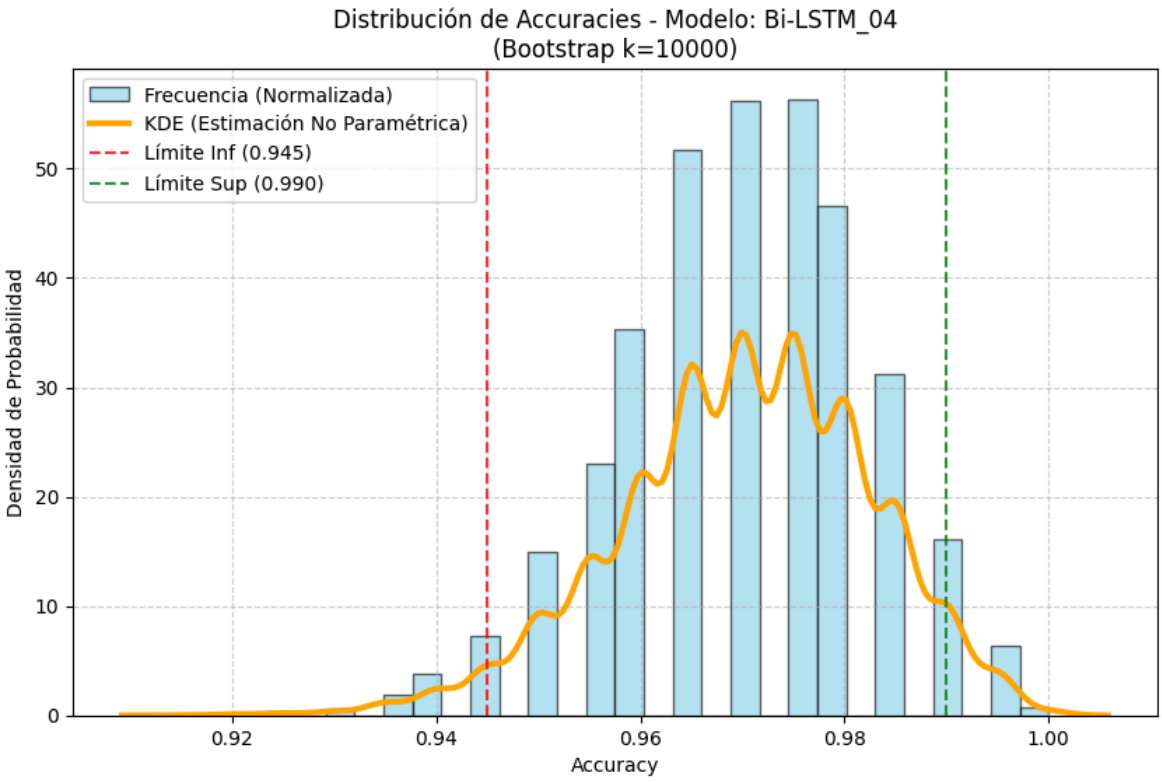
In [19]:
```
_, _ = ejecutar_experimento_individual(
    name='Bi-LSTM_04',
    lstm_layers=[128, 64],
    dense_layers=[64, 32],
    dropout=0.3,
    lr=0.00005,
    opt_name='adam',
    bidirectional=True,
    patience=20,
    epochs=120
)
```

```
Iniciando: Bi-LSTM_04
Arq: LSTM=[128, 64] (Bi=True) | Dense=[64, 32]
Params: Opt=adam | LR=5e-05 | Drop=0.3 | Patience=20
------------------------------------------------------------
GroupKFold Keras (5 splits)
(Grupos detectados: 100)
============================================================

Pliegue 1/5
/opt/miniconda3/envs/exml-py310/lib/python3.10/site-packages/keras/src/layers/core/masking.py:48: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

```
Fin Fold 1. Acc Val: 0.9500 | Loss: 0.1942

Pliegue 2/5
Fin Fold 2. Acc Val: 0.9500 | Loss: 0.0742

Pliegue 3/5
Fin Fold 3. Acc Val: 1.0000 | Loss: 0.0250

Pliegue 4/5
Fin Fold 4. Acc Val: 0.9500 | Loss: 0.1735

Pliegue 5/5
Fin Fold 5. Acc Val: 1.0000 | Loss: 0.0409
============================================================
Resultados GroupKFold:
Accuracy Medio: 0.9700
Desviación Std: 0.0245
Calculando Bootstrap (10,000 iteraciones)
Accuracy Promedio (Bootstrap): 0.9701
Intervalo de Confianza 95%: [0.9450, 0.9900]
Resultado: Acc Promedio 0.9701 | IC 95%: [0.9450, 0.9900]

Curvas de aprendizaje promedio:
Graficando las primeras 72 épocas comunes
```







Experimentación sin aumentación con Landmarks

Para esta experimentación, adaptaremos nuestra función de ejecutar expermientos para no modificarla

In [ ]:
```
import numpy as np

# Cargamos los datos
X_correct_original = np.load('../Data/Processed/correct_push_ups_landmarks.npy')
X_incorrect_original = np.load('../Data/Processed/wrong_push_ups_landmarks.npy')

# Preparamos datos de prueba sin espejos
X_test_original = np.vstack((X_correct_original, X_incorrect_original))
y_test_original = np.array([1]*len(X_correct_original) + [0]*len(X_incorrect_ori

# Grupos simples (Cada video es un grupo porque no hay espejos)
groups_original = np.arange(len(X_test_original))

print("Datos para prueba sin espejos preparados:")
print(f"X: {X_test_original.shape}")
print(f"y: {y_test_original.shape}")
```

```
🧪 Datos para prueba SIN ESPEJOS preparados:
   X: (100, 160, 66)
   y: (100,)
```

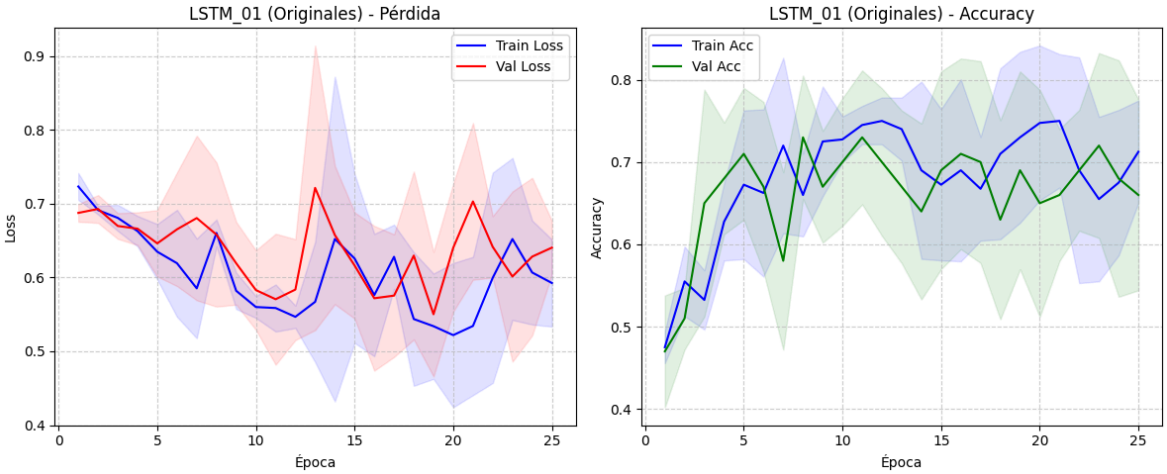
In [ ]:
```
def ejecutar_experimento_sin_aumentación(name, lstm_layers, dense_layers, dropou
                                         opt_name, patience, bidirectional=False
                                         epochs=40, batch_size=16):

    print(f"Test de solo originales (sin data augmentation)")
    print(f"Iniciando: {name}")
    print("-" * 60)
```

```
    # Builder (Igual)
    model_builder = lambda t, f: create_custom_lstm(
        t, f,
        lstm_layers=lstm_layers,
        dense_layers=dense_layers,
        dropout_rate=dropout,
        learning_rate=lr,
        optimizer_name=opt_name,
        bidirectional=bidirectional
    )

    # Aqui está el unico cambio importante
    # En vez de usar X_total, forzamos el uso de X_test_original
    mean_acc, std_acc, histories, final_y_true, final_y_pred = run_kfold_experim
        model_fn=model_builder,
        X_train=X_test_original, # Datos limpios
        y_train=y_test_original, # Etiquetas limpias
        groups=groups_original, # Grupos simples
        epochs=epochs, batch_size=batch_size, n_splits=5, patience=patience
    )

    # Gráficas
    print("\nCurvas de aprendizaje (Solo Originales)")
    plot_kfold_history_keras(histories, title=f'{name} (Originales)')

    # Bootstrap
    print(f"Calculando Bootstrap")
    lower, upper, stats = calcular_bootstrap_intervalo(final_y_true, final_y_pre
    boot_mean = np.mean(stats)

    # Guardamos en la tabla GLOBAL
    global results_table
    # Añadimos etiqueta al nombre para diferenciarlo en la tabla
    name_tagged = name + "_SOLO_ORIGINAL"

    results_table.append({
        'Modelo': name_tagged,
        'Dataset': 'SOLO_ORIGINAL',
        'Acc CV': mean_acc,
        'Acc Boot': boot_mean,
        'IC 95%': f"[{lower:.3f}-{upper:.3f}]",
        'LSTM Layers': str(lstm_layers),
        'Bidirectional': bidirectional,
        'Dense Layers': str(dense_layers),
        'Optimizador': opt_name,
        'Batch_size' : batch_size,
        'Epochs': epochs,
        'LR': lr,
        'Dropout': dropout,
        'Patience': patience
    })

    print(f"Guardado como '{name_tagged}'. Acc: {boot_mean:.4f}")

    # 3. Guardamos
    filename = f"../results/bootstrap_accuracies_exp_{name}.npy"
    np.save(filename, stats)

    return final_y_true, final_y_pred
```

Exp-LSTM-01

In [ ]:
```
_, _ = ejecutar_experimento_sin_aumentación(
    name='LSTM_01',
    lstm_layers=[64],
    dense_layers=[32],
    bidirectional=False,
    dropout=0.2,
    lr=0.001,
    opt_name='adam',
    patience=15
)
```

```
Test de solo originales (sin data augmentation)
Iniciando: LSTM_01
------------------------------------------------------------
GroupKFold Keras (5 splits)
(Grupos detectados: 100)
============================================================

Pliegue 1/5
/opt/miniconda3/envs/exml-py310/lib/python3.10/site-packages/keras/src/layers/core/masking.py:48: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
Fin Fold 1. Acc Val: 0.9000 | Loss: 0.3718

Pliegue 2/5
Fin Fold 2. Acc Val: 0.7000 | Loss: 0.5738

Pliegue 3/5
Fin Fold 3. Acc Val: 0.8000 | Loss: 0.4749

Pliegue 4/5
Fin Fold 4. Acc Val: 0.8500 | Loss: 0.4875

Pliegue 5/5
Fin Fold 5. Acc Val: 0.8000 | Loss: 0.4229
============================================================
Resultados GroupKFold:
Accuracy Medio: 0.8100
Desviación Std: 0.0663

Curvas de aprendizaje (Solo Originales)
Graficando las primeras 25 épocas comunes
```



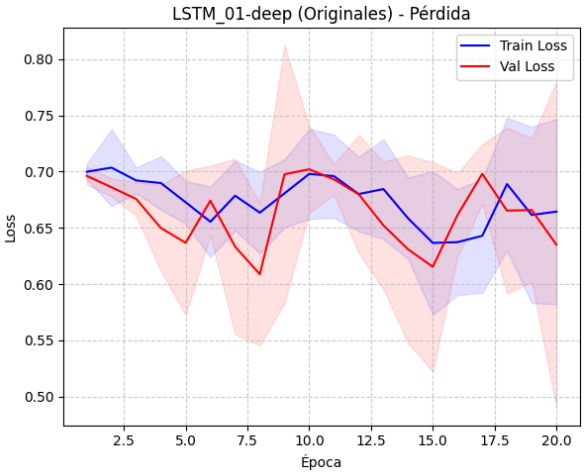

```
Calculando Bootstrap
Accuracy Promedio (Bootstrap): 0.8095
Intervalo de Confianza 95%: [0.7300, 0.8800]
Guardado como 'LSTM_01_SOLO_ORIGINAL'. Acc: 0.8095
```

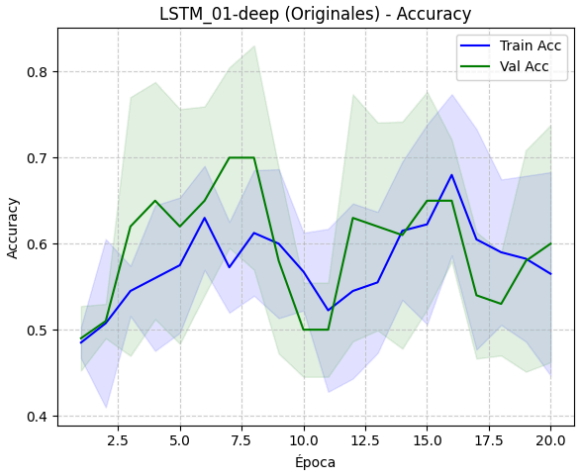
Exp-LSTM-01-deep

In [ ]:
```python
_, _ = ejecutar_experimento_sin_aumentación(
    name='LSTM_01-deep',
    lstm_layers=[128, 64],
    dense_layers=[64, 32],
    bidirectional=False,
    dropout=0.2,
    lr=0.001,
    opt_name='adam',
    patience=15,
    epochs = 100
)
```

```
Test de solo originales (sin data augmentation)
Iniciando: LSTM_01-deep
------------------------------------------------------------
GroupKFold Keras (5 splits)
(Grupos detectados: 100)
============================================================

Pliegue 1/5
/opt/miniconda3/envs/exml-py310/lib/python3.10/site-packages/keras/src/layers/core/masking.py:48: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

```
Fin Fold 1. Acc Val: 0.7500 | Loss: 0.5793

Pliegue 2/5
Fin Fold 2. Acc Val: 0.7500 | Loss: 0.5179

Pliegue 3/5
Fin Fold 3. Acc Val: 0.7000 | Loss: 0.6270

Pliegue 4/5
Fin Fold 4. Acc Val: 0.9000 | Loss: 0.3342

Pliegue 5/5
Fin Fold 5. Acc Val: 0.8000 | Loss: 0.5358
============================================================
Resultados GroupKFold:
Accuracy Medio: 0.7800
Desviación Std: 0.0678

Curvas de aprendizaje (Solo Originales)
Graficando las primeras 20 épocas comunes
```





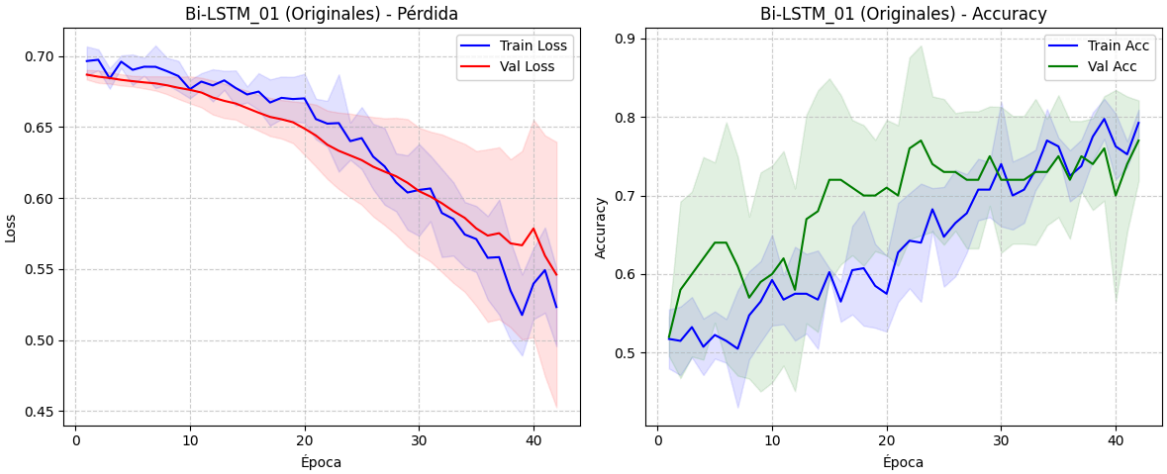

```
Calculando Bootstrap
Accuracy Promedio (Bootstrap): 0.7808
Intervalo de Confianza 95%: [0.7000, 0.8600]
Guardado como 'LSTM_01-deep_SOLO_ORIGINAL'. Acc: 0.7808
```

Exp-Bi-LSTM-01

In [24]:

```
_, _ = ejecutar_experimento_sin_aumentación(
    name='Bi-LSTM_01',
    lstm_layers=[128, 64],
    dense_layers=[64, 32],
    dropout=0.3,
    lr=0.00005,
    opt_name='adam',
    bidirectional=True,
    patience=20,
    epochs=120
)
```

```
Test de solo originales (sin data augmentation)
Iniciando: Bi-LSTM_01
------------------------------------------------------------
GroupKFold Keras (5 splits)
(Grupos detectados: 100)
============================================================

Pliegue 1/5
/opt/miniconda3/envs/exml-py310/lib/python3.10/site-packages/keras/src/layers/core/masking.py:48: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
Fin Fold 1. Acc Val: 0.7500 | Loss: 0.5414

Pliegue 2/5
Fin Fold 2. Acc Val: 0.8000 | Loss: 0.3848

Pliegue 3/5
Fin Fold 3. Acc Val: 0.6000 | Loss: 0.6686

Pliegue 4/5
Fin Fold 4. Acc Val: 0.9000 | Loss: 0.3067

Pliegue 5/5
Fin Fold 5. Acc Val: 0.8000 | Loss: 0.4435
============================================================
Resultados GroupKFold:
Accuracy Medio: 0.7700
Desviación Std: 0.0980

Curvas de aprendizaje (Solo Originales)
Graficando las primeras 42 épocas comunes
```




```
Calculando Bootstrap
Accuracy Promedio (Bootstrap): 0.7700
Intervalo de Confianza 95%: [0.6900, 0.8500]
Guardado como 'Bi-LSTM_01_SOLO_ORIGINAL'. Acc: 0.7700
```

Experimentación sin aumentación con Ángulos

In [25]:
```python
import numpy as np

# Cargar los datos
X_correct_original = np.load('../Data/Processed/correct_push_ups_angles.npy')
```

```
X_incorrect_original = np.load('../Data/Processed/wrong_push_ups_angles.npy')

X_test_original = np.vstack((X_correct_original, X_incorrect_original))
y_test_original = np.array([1]*len(X_correct_original) + [0]*len(X_incorrect_ori

# Grupos simples
groups_original = np.arange(len(X_test_original))

print("Datos para prueba sin espejos preparados:")
print(f"X: {X_test_original.shape}")
print(f"y: {y_test_original.shape}")
```

```
Datos para prueba sin espejos preparados:
X: (100, 160, 10)
y: (100,)
```

Exp-LSTM-02

In [26]:

```
_, _ = ejecutar_experimento_sin_aumentación(
    name='LSTM_02',
    lstm_layers=[64],
    dense_layers=[32],
    dropout=0.2,
    lr=0.00005,
    opt_name='adam',
    patience=15,
    epochs=100
)
```

```
Test de solo originales (sin data augmentation)
Iniciando: LSTM_02
------------------------------------------------------------
GroupKFold Keras (5 splits)
(Grupos detectados: 100)
============================================================

Pliegue 1/5
/opt/miniconda3/envs/exml-py310/lib/python3.10/site-packages/keras/src/layers/cor
e/masking.py:48: UserWarning: Do not pass an `input_shape`/`input_dim` argument t
o a layer. When using Sequential models, prefer using an `Input(shape)` object as
the first layer in the model instead.
  super().__init__(**kwargs)
```

```
Fin Fold 1. Acc Val: 0.9500 | Loss: 0.4711

Pliegue 2/5
Fin Fold 2. Acc Val: 0.9000 | Loss: 0.5256

Pliegue 3/5
Fin Fold 3. Acc Val: 0.8500 | Loss: 0.5951

Pliegue 4/5
Fin Fold 4. Acc Val: 0.9000 | Loss: 0.5027

Pliegue 5/5
Fin Fold 5. Acc Val: 0.8000 | Loss: 0.6289
============================================================
Resultados GroupKFold:
Accuracy Medio: 0.8800
Desviación Std: 0.0510

Curvas de aprendizaje (Solo Originales)
Graficando las primeras 100 épocas comunes
```